



Escuela
Politécnica
Superior

Practical guide to implement OpenRMF in multi-robot environments

Grado en Ingeniería Informática



Trabajo Fin de Grado

Autor:

Joel Romero Mollá

Tutor/es:

María del Pilar Arques Corrales

Julio 2024



Universitat d'Alacant
Universidad de Alicante

Abstract

The rapid advancements in robotics technology have significantly contributed to the development and deployment of multi-robot systems (MRSs), which offer numerous advantages over traditional single-robot systems. These advantages include enhanced operational effectiveness, efficiency, flexibility, fault tolerance, and the ability to leverage the specialized capabilities of diverse robots for collaborative task execution. This project presents a practical guide to implementing the Robotics Middleware Framework (OpenRMF) in multi-robot environments, addressing the critical challenges of coordination, pathfinding, and communication among multiple robots.

OpenRMF is a robust middleware solution designed to simplify the integration and management of heterogeneous robot fleets within predefined environments. It supports the seamless operation of various robot models, enabling efficient coordination and communication essential for executing complex tasks collaboratively. This guide provides comprehensive instructions for installing and configuring OpenRMF, creating a project structure akin to RMF demos, and setting up dependencies in `package.xml` and `CMakeLists.txt` files. Emphasis is placed on ensuring proper system localization to guarantee compatibility and optimal performance of the framework.

The implementation process begins with the preparation of the environment, including verifying locale settings to support UTF-8 encoding and installing essential utilities and dependencies. Detailed steps for setting up ROS 2 (Robot Operating System), which forms the backbone of OpenRMF, are provided. The transition from ROS 1 to ROS 2 is highlighted, showcasing the improvements in real-time communication, security, and flexibility that ROS 2 offers, making it better suited for multi-robot systems.

Key aspects of the guide include the setup of the RMF Web for visualization and control, which facilitates monitoring and managing robot operations from a web interface. Users are guided through creating and configuring various scenarios using OpenRMF, including defining launch parameters, establishing robot fleets, and configuring robot behaviors. The guide also addresses the challenges associated with multi-robot coordination, such as collision avoidance, dynamic replanning, task allocation, and synchronization.

Performance evaluation of OpenRMF is a critical component of the project, involving experiments to test its capabilities in handling coordination, pathfinding, and communication tasks. These evaluations aim to demonstrate OpenRMF's effectiveness in managing multi-robot operations and highlight its strengths and potential areas for improvement. The results of these evaluations provide valuable insights into the practical applicability of OpenRMF in real-world scenarios, ensuring that the framework meets the demands of diverse applications, from industrial automation to disaster response.

By following this guide, users will gain a thorough understanding of OpenRMF's architecture and capabilities, enabling them to establish a functional RMF installation that supports the development and deployment of sophisticated multi-robot systems. The project contributes to the broader adoption and optimization of OpenRMF, facilitating the advancement of the field of multi-robot systems and enhancing their deployment in various industries. This work underscores the importance of robust middleware solutions like OpenRMF in overcoming the inherent challenges of multi-robot coordination and communication, paving the way for more efficient and effective robotic operations.

Justification and Objectives

The rapid advancement in robotics technology has led to the development of multi-robot systems (MRSs), which offer significant advantages over traditional single-robot systems. Multi-robot systems enhance operational effectiveness, efficiency, flexibility, fault tolerance, and diversity by enabling collaborative task execution and leveraging the specialized capabilities of diverse robots. Despite these advantages, there are several challenges in coordinating, pathfinding, and communication among multiple robots, which necessitates the use of robust middleware solutions.

OpenRMF (Robotics Middleware Framework) is a promising middleware solution designed to address these challenges. It simplifies the integration of heterogeneous robot models and improves operational efficiency within predefined environments. Given the increasing complexity and scale of applications involving multi-robot systems, the implementation and evaluation of OpenRMF are crucial for advancing the field and ensuring the seamless operation of multi-robot fleets in various industries.

This project aims to provide a comprehensive guide for the implementation and evaluation of OpenRMF in multi-robot systems. By offering detailed installation instructions, scenario setups, and performance evaluations, this work will contribute to the broader adoption and optimization of OpenRMF in real-world applications.

The main objective is to implement and evaluate OpenRMF as a middleware solution for enhancing the coordination and management of multi-robot systems.

Specific Objectives:

1. Installation and Configuration:

- To provide detailed, step-by-step instructions for installing and configuring OpenRMF.
- To ensure the proper setup of the system by verifying the installation through example scenarios.

2. Scenario Implementation:

- To design and set up various scenarios using OpenRMF, including the creation of configuration files, definition of launch parameters, and establishment of robot fleets.
- To demonstrate OpenRMF's capability to handle different multi-robot configurations and tasks.

3. Performance Evaluation:

- To conduct experiments testing OpenRMF's performance in handling coordination, pathfinding, and communication challenges.
- To observe and confirm OpenRMF's effectiveness in managing multi-robot operations and highlight its strengths.

By achieving these objectives, the project aims to establish OpenRMF as a reliable and efficient middleware solution for multi-robot systems, facilitating their deployment and operation in various industrial and research contexts.

Acknowledgements

I would like to express my deepest gratitude to all those who have supported and guided me throughout the completion of this project.

Firstly, I would like to thank my advisor, María del pilar Arques Corrales, for their invaluable guidance, encouragement, and insightful feedback throughout the duration of this project. Their expertise and support have been instrumental in shaping the direction and quality of this work.

I am also grateful to my professors and mentors at University of Alicante, whose knowledge and teachings have provided me with the foundational skills and understanding necessary to undertake this project.

A special thanks to my family and friends for their unwavering support, patience, and encouragement during this challenging journey. Their belief in my abilities has been a constant source of motivation.

Additionally, I would like to acknowledge the contributions of the OpenRMF community and developers, whose work and documentation have been essential resources for this project.

Finally, I extend my gratitude to all my colleagues and peers who have offered their support and shared their knowledge, making this academic journey a collaborative and enriching experience.

Thank you all for your invaluable contributions and encouragement.

Dedication

I dedicate this work to my family and friends, whose support and encouragement have been a constant source of strength and inspiration.

A special mention goes to those who have helped me stay focused and motivated throughout this journey, providing the guidance and support I needed to see this project through to completion.

To all those who believed in me and contributed to my academic and personal growth, this work is a testament to your unwavering faith and encouragement. Thank you for being there every step of the way.

Contents

1. Introduction	4
2. State of the art	6
2.1. Multi Robot Systems	6
2.2. Middlewares for MRSs	6
2.3. ROS	7
2.4. Improvements in ROS 2	8
2.5. Tools and dependencies	9
2.5.1. Gazebo	9
2.5.2. Python	9
2.5.3. RViz	10
2.5.4. FastAPI and Uvicorn	10
2.5.5. Flask-SocketIO	10
2.6. Challenges	11
2.6.1. Robot Teamwork	11
2.6.2. Multi-Agent Pathfinding	11
2.6.3. Communication	12
2.6.4. Coordination	12
2.6.5. Cooperation	12
2.6.6. Uncertainty	13
2.7. What OpenRMF can help with	13
3. Objectives	14
3.1. General Objective	14
3.2. Specific Objectives	14
4. Installation of RMF	15
4.1. Check Localization	15
4.1.1. Verify Locale Settings	15
4.1.2. Generate UTF-8 Locale	15
4.2. Base install	16
4.2.1. SocketIO Issue and Fix	17
4.3. ROS extra configuration	17
4.4. Packages Required for RMF Demo	18
4.5. Dependency Manager	18
4.6. Install RMF Web	19
4.7. Installation check	21
4.7.1. ROS 2 Core	21
4.7.2. ROS 2 RMF	21

5. First project	24
5.1. Create Project Structure and Basic Configuration	24
5.1.1. Copy and Create the Following Packages	24
5.1.2. Set Up Dependencies and Project Configuration	26
5.2. Configure scenario	28
5.2.1. Project config	29
5.2.2. Project simulation	34
5.3. Create a Map	37
5.4. Execute tasks	50
6. Large project Example	53
7. Conclusion	60
8. Improvements	61
8.1. In depth guide	61
8.2. Generating Your Own Robot Model and Specification	61
8.3. Adding Existing Robots	61
8.4. Adding Robots with Object-Carrying Capabilities	62
8.5. Exploring Advanced Scenarios	62

List of Figures

1	Project Structure	24
2	Launch files composition structure	28
3	Creating a New Level	38
4	Adding Walls	38
5	Aligning Walls	39
6	Adding a Floor	40
7	Spawning a Robot	41
8	Robot Configuration	41
9	Adding a Lane	42
10	First Launch	43
11	Setting Measurements	43
12	Measurement Simulation	44
13	Adding a Door	45
14	Door Simulation	46
15	Select a model	47
16	Place models	48
17	Simulation with models	48
18	Second Floor	49
19	Creating a Lift	50
20	Simple API server displaying a list of robots	51
21	Robot moving after executing a task	52
22	EPS1 Building ground floor	53
23	EPS1 Building first floor	54
24	EPS1 Building second floor	55
25	EPS1 Building	56
26	First row of chairs	57
27	Contrary row of chairs	57
28	Yaml file with chairs highlighted	58
29	Chairs aligned	59
30	Final result	59

1. Introduction

The field of robotics has experienced significant advancements in recent decades, leading to innovative developments that have revolutionized various industries. Among these advancements, the emergence of multi-robot systems (MRS) is particularly noteworthy. Unlike traditional single-robot systems, MRS provides numerous advantages, such as enhanced effectiveness, efficiency, flexibility, fault tolerance, and the ability to execute complex tasks collaboratively. These benefits underscore the relevance and importance of MRS in today's rapidly evolving technological landscape.

The significance of multi-robot systems is highlighted by their increasing application across a wide range of sectors, including industrial automation, surveillance, exploration, and disaster response. In industrial environments, for example, the deployment of multiple robots can streamline manufacturing processes, improve precision, and reduce downtime. In surveillance and exploration, MRS enhances coverage and data collection, providing more comprehensive insights than single robots. Furthermore, in disaster response scenarios, the collaborative efforts of multiple robots can significantly improve the efficiency and effectiveness of rescue operations, ensuring quicker and safer interventions.

Despite these advantages, the deployment and management of multi-robot systems present substantial challenges. One of the primary challenges is the coordination and communication among multiple robots, particularly when these robots are from different manufacturers with varying communication protocols and specifications. Ensuring seamless interoperability and efficient task allocation in such heterogeneous environments is a complex problem that necessitates robust and sophisticated solutions. Additionally, pathfinding and real-time decision-making in dynamic and uncertain environments add layers of complexity to the operation of MRS.

The Robotics Middleware Framework (OpenRMF) emerges as a promising solution to these challenges. OpenRMF is designed to facilitate the integration and coordination of heterogeneous robot fleets, offering a robust middleware solution that simplifies communication, coordination, and task management. The framework's ability to support various robot models and streamline their operations within predefined environments makes it an invaluable tool for researchers and developers working with multi-robot systems.

The primary purpose of this research is to explore and demonstrate the effectiveness of OpenRMF as a middleware solution for enhancing the coordination and management of multi-robot systems. This study aims to provide a practical guide for the installation, configuration, and deployment of OpenRMF, thereby facilitating its adoption and optimization in real-world applications. By offering detailed instructions and performance evaluations, this research seeks to contribute to the broader understanding and utilization of OpenRMF, addressing the critical challenges associated with multi-robot coordination and communication.

The relevance and timeliness of this research are evident in the growing need for efficient multi-robot systems across various industries. As the complexity and scale of applications involving MRS continue to increase, the demand for robust middleware solutions like OpenRMF becomes more pressing. This study not only aims to showcase the capabilities of OpenRMF but also to highlight its potential in driving advancements in multi-robot systems, thereby enhancing their deployment and operation in diverse contexts.

In conclusion, the investigation into OpenRMF and its implementation in multi-robot environments is of paramount importance. It addresses current technological challenges, provides practical solutions, and paves the way for future innovations in the field of robotics. By focusing on the installation, configuration, and evaluation of OpenRMF, this research aims to establish it as a reliable and efficient middleware solution, thereby contributing to the advancement of multi-robot systems and their applications in various industrial and research settings.

2. State of the art

In this section, we will explore the state of the art of multi-robot systems (MRSs). We will examine the advantages of MRSs over single robot systems, including effectiveness, efficiency, flexibility, fault tolerance, and diversity. This section will also discuss various middlewares used for MRS coordination and communication, highlighting the differences between event-based and publisher-subscriber communication protocols. Additionally, we will address the challenges faced by MRSs, such as teamwork, pathfinding, communication, coordination, cooperation, and dealing with uncertainty. Finally, we will introduce OpenRMF, explaining how it can assist in overcoming some of these challenges and facilitating the development and maintenance of multi-robot systems.

2.1. Multi Robot Systems

Multi-robot systems (MRSs) are a paradigm in robotics that involve coordinating multiple robots to accomplish tasks collaboratively. Unlike traditional single-robot approaches, MRSs offer several distinct advantages as shown in Roldán-Gómez et al. (2021).

- **Effectiveness:** Multiple robots can often perform tasks more quickly than a single robot by dividing the workload. For instance, distributing the area to be covered among multiple drones can significantly accelerate task completion.
- **Efficiency:** Specializing robots in different skills enhances mission efficiency. In search operations, combining aerial and ground robots allows for rapid surface scanning by drones followed by detailed ground-level analysis, leveraging the strengths of each robot type.
- **Flexibility:** MRSs can adapt to diverse situations through task allocation strategies and the heterogeneous capabilities of the robots involved. This adaptability ensures that the system can respond effectively to varying mission requirements.
- **Fault Tolerance:** Critical to MRSs is their ability to maintain mission integrity despite individual robot failures or incidents. If one robot malfunctions or encounters an obstacle, the remaining robots can continue the mission, ensuring robustness and continuity.
- **Diversity:** Leveraging robots specialized in different domains—such as land, sea, and air—enhances the system's ability to overcome mission-specific challenges. For example, if terrain conditions are hazardous, airborne robots can continue operations while ground-based robots may be hindered.

These advantages highlight the potential of multi-robot systems to enhance operational efficiency, flexibility, and resilience across a wide range of applications, from surveillance and exploration to disaster response and industrial automation.

2.2. Middlewares for MRSs

Middlewares play a crucial role in the coordination and communication of MRSs and are a key feature for the future improvement of multi robot systems, some of this middlewares are analyzed in Kurt (2020). They can be broadly classified into two categories based on their communication protocols: event-based and publisher-subscriber.

Event-Based Coordination: Middlewares like Miro, Orocos, and CLARAty utilize event-based communication, often relying on RPC protocols. This method requires some knowledge of the other end and may not be sufficient for decoupling in heterogeneous multi-robot systems. For instance:

- Orocos and Miro rely on heavyweight architectures like CORBA and IceRPC, respectively.
- CLARAty uses a custom communication protocol developed for NASA rovers, capable of handling unreliable channels and supporting both centralized and decentralized modes.

Publisher-Subscriber Communication: More commonly used middlewares such as RoboFrame, Spica, and ROS utilize publisher-subscriber protocols:

- RoboFrame and Spica handle unreliable communication and distributed computation.
- ROS 1 initially used a custom communication protocol, which was not well-suited for MRSs until ROS 2 was introduced. ROS 2 employs the widely known Data Distribution Service (DDS), which significantly enhances its ability to support multi-robot systems.

ROS (Robot Operating System) is a widely adopted framework in robotics research and industry. Its extensive use and active community make it a robust choice for developing multi-robot systems. ROS provides a standardized set of tools, libraries, and conventions, facilitating the development, simulation, and deployment of complex robotic applications. The transition from ROS 1 to ROS 2 further improved its capabilities, making it more suitable for MRSs with better real-time communication, enhanced security, and more flexible middleware options.

2.3. ROS

The Robot Operating System (ROS) is a powerful and flexible framework designed for writing robot software. Initially developed by Willow Garage and Stanford's Artificial Intelligence Laboratory, ROS has become a cornerstone in the field of robotics, thanks to its extensive open-source community and versatile capabilities. The main goal of ROS is to provide a standard for robotic software development, enabling researchers and developers to build, share, and reuse code efficiently.

At its core, ROS operates on a graph architecture where processes are represented as nodes that can communicate with each other. This architecture facilitates the modular design of robotic systems, allowing for the integration of various components and functionalities. The main components of ROS include:

- **Nodes:** Nodes are the fundamental building blocks of ROS applications. Each node is an executable that performs a specific task and communicates with other nodes to achieve complex robotic behaviors. Nodes can be written in different programming languages, such as C++ or Python.
- **Topics:** Topics provide a flexible and scalable way for nodes to exchange messages. A topic is a named bus over which nodes send and receive messages. This many-to-many communication model allows multiple nodes to publish to or subscribe to the same topic, facilitating data sharing and coordination.
- **Messages:** Messages are the data structures used by nodes to communicate. They consist of typed fields and can include primitive data types (e.g., integers, floats) as well as arrays of these types. The simplicity

of message structures ensures efficient and reliable data exchange.

- **Services:** While topics are suitable for continuous data streams, services provide a request-response communication model. Services are used for one-time interactions, where a node sends a request and waits for a response. This synchronous communication is essential for tasks that require immediate feedback.
- **Packages:** Packages are the organizational units of ROS software. A package can contain nodes, libraries, datasets, configuration files, and other resources. The package structure promotes reusability and modularity, allowing developers to easily share and integrate their work with others.

2.4. Improvements in ROS 2

ROS 2 is the next-generation version of the Robot Operating System, designed to address the limitations and challenges encountered in ROS 1. Building on the lessons learned from its predecessor, ROS 2 introduces several key improvements that enhance its performance, scalability, and flexibility, some of these changes are stated in Dirk Thomas (2017).

- **Middleware Abstraction Layer:** One of the most significant advancements in ROS 2 is the adoption of the Data Distribution Service (DDS) as its middleware. DDS provides a robust framework for communication, offering better support for real-time systems, improved reliability, and greater flexibility in handling node interactions.
- **Real-time Capabilities:** ROS 2 brings enhanced support for real-time systems, a critical requirement for many robotic applications. This improvement ensures that ROS 2 can handle the deterministic behavior needed for real-time processing, making it suitable for time-sensitive tasks.
- **Enhanced Security:** Security is a critical aspect of any robotic system. ROS 2 incorporates DDS-Security, which includes features like authentication, encryption, and access control. These security measures protect communications between nodes, ensuring data integrity and confidentiality.
- **Improved Modularity:** ROS 2's architecture is more modular than that of ROS 1. This modularity allows developers to use only the components they need, reducing system overhead and improving performance. It also simplifies the process of extending and customizing the framework.
- **Cross-platform Support:** While ROS 1 primarily supported Unix-based systems, ROS 2 is designed to be cross-platform, with compatibility for Unix-based systems, Windows, and real-time operating systems (RTOS). This cross-platform support enhances the versatility and applicability of ROS 2 in various environments.
- **Lifecycle Management:** ROS 2 introduces managed nodes with a defined lifecycle, providing a standardized way to manage node states. This feature helps developers control the startup, shutdown, and error handling processes more effectively, ensuring stable and predictable system behavior.
- **Quality of Service (QoS) Policies:** ROS 2 allows fine-grained control over communication behaviors through Quality of Service settings. These policies enable developers to tune communication parameters

to meet specific application requirements, such as ensuring data delivery deadlines or optimizing resource usage.

In summary, ROS 2 represents a substantial evolution from ROS 1, addressing key areas such as middleware robustness, real-time processing, security, modularity, cross-platform support, lifecycle management, and communication quality. These enhancements make ROS 2 a more powerful and flexible framework, capable of meeting the demands of modern robotic applications and paving the way for future innovations in the field.

2.5. Tools and dependencies

In addition to ROS, several other tools and dependencies are essential for the successful implementation and operation of OpenRMF in multi-robot environments. These tools enhance various aspects of development, simulation, visualization, and system management. Key tools include Gazebo, a high-fidelity simulation environment for testing robotic applications in complex settings; Python, a versatile programming language integral to developing ROS-based applications; and RViz, a powerful visualization tool that provides real-time insights into robot states and environments. Together, these tools and dependencies ensure the robustness and efficiency of multi-robot systems, facilitating their development, testing, and deployment.

2.5.1. Gazebo

Gazebo is a high-fidelity robot simulation environment that enables the testing and development of robotic applications in complex and dynamic environments. It provides realistic rendering of environments and advanced physics simulation, making it an essential tool for developing and testing multi-robot systems. Gazebo supports a wide range of sensors and actuators, allowing for detailed and accurate simulations of real-world scenarios. The integration of Gazebo with ROS allows for seamless interaction between simulated robots and ROS-based control systems, enabling comprehensive testing and validation before deploying robots in real environments.

Key Features:

- Realistic environment and physics simulation
- Support for a wide range of sensors and actuators
- Integration with ROS for synchronized simulation and control
- Extensive library of models and environments

2.5.2. Python

Python is a versatile and widely-used programming language that plays a central role in the development of ROS-based applications. Its simplicity and readability make it an ideal choice for scripting and automation tasks within the ROS ecosystem. Python's extensive library support and community-driven development provide a robust foundation for implementing complex algorithms and data processing tasks. In the context of OpenRMF, Python is used for writing nodes, developing algorithms for coordination and pathfinding, and managing communication between different components of the multi-robot system.

Key Features:

- Simple and readable syntax
- Extensive standard libraries and third-party modules
- Strong community support
- Integration with ROS through rospy, the Python client library for ROS

2.5.3. RViz

RViz (ROS Visualization) is a powerful visualization tool that allows developers to visualize the state of a robot and its environment. It provides a graphical interface to display sensor data, robot models, and other relevant information in real-time. RViz is an essential tool for debugging and monitoring the behavior of robots, as it enables developers to see how robots perceive their environment and execute tasks. The ability to customize the display with different types of markers and overlays makes RViz a versatile tool for developing and testing multi-robot systems.

Key Features:

- Real-time visualization of sensor data and robot states
- Customizable displays with various markers and overlays
- Integration with ROS for seamless data visualization
- Support for multiple data sources and visualization plugins

2.5.4. FastAPI and Uvicorn

FastAPI is a modern web framework for building APIs with Python, known for its high performance and ease of use. Uvicorn is a lightning-fast ASGI server implementation, used to run FastAPI applications. Together, FastAPI and Uvicorn are used in OpenRMF to develop RESTful APIs for interacting with the multi-robot system. These tools enable the creation of efficient and scalable web services, facilitating communication and control within the multi-robot environment.

Key Features:

- High performance and ease of use
- Support for asynchronous programming
- Comprehensive documentation and community support
- Integration with other Python-based tools and frameworks

2.5.5. Flask-SocketIO

Flask-SocketIO is an extension for Flask that enables WebSocket communication in Python applications. It is used in OpenRMF to implement real-time communication between the web interface (RMF Web) and the backend system. This real-time communication is essential for providing up-to-date information on robot status, task progress, and other dynamic data within the multi-robot environment.

Key Features:

- Real-time communication via WebSockets

- Easy integration with Flask applications
- Support for multiple transport protocols
- Comprehensive event handling and messaging capabilities

2.6. Challenges

The interest in multi-robot systems (MRS) is increasing thanks to the previously mentioned advantages, but there are still some challenges that we need to solve to make MRSs a viable solution. Current and future middlewares should address these issues to improve their usability and functionality. As explained in Jesús Roldán-Gómez et al. (2021) and Meseguer Valenzuela et al. (2023), some of the key challenges are:

2.6.1. Robot Teamwork

Ensuring seamless collaboration and coordination among multiple robots is critical. This involves developing algorithms and strategies that enable robots to work together effectively without human intervention. Challenges in robot teamwork include:

- **Task Allocation:** Distributing tasks among robots based on their capabilities, locations, and current states. Effective task allocation ensures that the workload is balanced and that each robot's unique capabilities are utilized optimally. Approaches like market-based mechanisms and auction systems have shown promise in dynamically assigning tasks to robots in real-time.
- **Synchronization:** Ensuring that robots perform tasks in a synchronized manner, avoiding conflicts and ensuring smooth operation. This is particularly challenging in dynamic environments where the tasks or conditions may change rapidly.
- **Resource Sharing:** Managing shared resources, such as charging stations, tools, or workspace areas, to prevent bottlenecks and conflicts. Techniques like resource reservation systems and shared resource scheduling are critical to managing these challenges effectively.

2.6.2. Multi-Agent Pathfinding

Developing efficient algorithms for path planning in dynamic environments with multiple agents is complex. Pathfinding challenges include:

- **Collision Avoidance:** Ensuring that robots do not collide with each other or with obstacles in the environment. Algorithms such as A* and D* have been adapted to multi-robot scenarios to enhance collision avoidance capabilities.
- **Dynamic Replanning:** Adapting paths in real-time based on changes in the environment, such as new obstacles or changing goals. This involves the use of dynamic path planning algorithms that can quickly re-evaluate and adjust paths as needed.
- **Scalability:** Creating algorithms that can handle a large number of robots simultaneously without a significant drop in performance. Scalability is achieved through techniques like hierarchical planning and decentralized control systems, which reduce computational load and improve response times.

2.6.3. Communication

Establishing reliable and scalable communication protocols for inter-robot and robot-to-infrastructure interactions is essential. Communication challenges include:

- **Latency:** Minimizing delays in communication to ensure real-time coordination and control. This requires the development of low-latency communication protocols that can handle high volumes of data transmission efficiently.
- **Bandwidth:** Managing the limited bandwidth available, especially in environments with many robots or high data transmission requirements. Techniques like data compression and prioritized messaging help optimize bandwidth usage.
- **Robustness:** Ensuring communication remains reliable despite interference, network congestion, or hardware failures. Redundant communication pathways and error-correction protocols are vital to maintaining robust communication networks.

2.6.4. Coordination

Synchronizing tasks and movements of multiple robots to achieve common goals requires sophisticated coordination mechanisms. Coordination challenges include:

- **Centralized vs. Decentralized Control:** Deciding between a centralized control system, which can become a bottleneck, and a decentralized approach, which can be more complex to implement. Centralized systems are easier to manage but can fail if the central node fails, whereas decentralized systems offer greater redundancy and fault tolerance.
- **Task Scheduling:** Optimally scheduling tasks to ensure timely completion while balancing the load across all robots. Advanced scheduling algorithms and AI-based approaches can help in dynamically scheduling tasks based on real-time data.
- **Prioritization:** Determining the priority of tasks and ensuring critical tasks are completed first. This involves developing strategies for dynamically adjusting task priorities based on changing conditions and requirements.

2.6.5. Cooperation

Facilitating cooperation among robots with different capabilities and roles is crucial for multi-robot systems. Cooperation challenges include:

- **Heterogeneous Teams:** Managing teams of robots with different functionalities, such as drones working with ground robots. Interoperability standards and modular software architectures are essential for integrating diverse robotic systems.
- **Interdependency:** Handling tasks that require multiple robots to work together, such as transporting a large object. This requires coordination algorithms that can manage dependencies and ensure that all necessary robots are available and synchronized for cooperative tasks.
- **Adaptability:** Ensuring robots can adapt their behavior based on the actions of other robots in the team.

Machine learning techniques and adaptive control systems can help robots learn and adjust their behaviors in real-time based on feedback from their environment and other robots.

2.6.6. Uncertainty

Handling uncertainties and dynamic changes in the environment effectively is a major challenge. Uncertainty challenges include:

- **Environmental Changes:** Adapting to changes in the environment, such as moving obstacles, varying lighting conditions, or changing weather. Robust perception systems and adaptive planning algorithms are critical for managing these dynamic changes.
- **Sensor Noise:** Dealing with inaccuracies and noise in sensor data, which can affect perception and decision-making. Sensor fusion techniques and noise filtering algorithms can help improve the reliability of sensor data.
- **Unpredictable Events:** Managing unexpected events, such as robot failures, power outages, or sudden changes in task requirements. Resilient system architectures and fault-tolerant algorithms are essential for ensuring that the robot fleet can continue operating effectively despite these disruptions.

2.7. What OpenRMF can help with

OpenRMF can help with some of these challenges, like coordination; any task added to the system is assigned to a fleet through a process of bidding, where every fleet manager bids an amount depending on the proximity, charge and any other property that could impact the task. It can also help with path finding, but RMF's version is reduced to a set of limited routes that are predefined by the user when creating the map layout, any other more complex implementation needs to be done by the user. By using the traffic editor included with RMF, problems like collisions or resource access are solved by the system without any intervention of the user.

The communication in RMF can be done in many different ways, since RMF is based on ROS 2 it can use the topic-based communication approach but it is not restricted to it, other methods such as sockets, TCP/IP, HTTP and more can be used. The ease of use that comes with RMF's coordination, path finding and communication systems makes it possible to implement more complex robot teamwork algorithms using the abstractions provided by RMF.

Multi-robot systems are hard to develop and maintain, there is the need to have different types of robots and each one can be from different manufacturers, with different communication protocols and different specifications. Using RMF as a middleware enables the development of complex solution by facilitating the integration of various robot models, where every robot requires a fleet adapter to be developed in order to work with RMF, once the adapter is done the robot can be used inside any RMF project.

All these properties make RMF a very powerful tool to develop simple and complex multi-robot systems, but it is bound to a layout. This means that RMF cannot be used in random or dynamic places, it is bound to a building or predefined area.

3. Objectives

3.1. General Objective

The main goal is to explore and demonstrate the effectiveness of the Robotics Middleware Framework (RMF) as a middleware solution for the coordination and management of multi-robot systems (MRS), providing a practical guide for researchers and developers.

3.2. Specific Objectives

1. **Installation and Configuration:** Provide a detailed guide for the installation and configuration of RMF to establish a functional environment for multi-robot systems.
2. **Middleware Analysis:** Analyze various middleware solutions for MRS coordination and communication, focusing on the transition from ROS 1 to ROS 2.
3. **Scenario Implementation:** Create and configure scenarios using RMF, defining launch parameters, and setting up fleet and robot configurations.
4. **Evaluation of RMF Capabilities:** Assess how RMF addresses challenges in pathfinding, coordination, and communication within multi-robot systems, and explore its flexibility in integrating various robot models.

4. Installation of RMF

In this section, we will guide you through the installation process of RMF (Robotics Middleware Framework) and its dependencies. You will find step-by-step instructions on preparing your environment, installing necessary dependencies, and configuring your system for RMF that are provided from various sources in Open Robotics. This includes checking your system's localization settings, performing the base installation of RMF and its dependencies, and setting up the required packages for the RMF demo. We will also cover the installation of RMF Web for visualization and control purposes. By the end of this section, you will have a fully operational RMF installation ready for use.

4.1. Check Localization

To ensure ROS 2 functions properly, you need to verify that your system's environment is compatible with UTF-8 encoding. Here's how to check and configure your locale settings:

4.1.1. Verify Locale Settings

First, check your system's current locale settings:

```
$ locale
LANG=es_ES.UTF-8
...
```

If the LANG variable does not include the suffix UTF-8, you will need to generate it.

4.1.2. Generate UTF-8 Locale

If your system does not have UTF-8 you will need to update your package lists and install the `locales` package if it's not already installed:

```
$ sudo apt update && sudo apt install locales
```

Generate the required UTF-8 locales:

```
$ sudo locale-gen es_ES es_ES.UTF-8
```

Update the locale settings:

```
$ sudo update-locale LC_ALL=es_ES.UTF-8 LANG=es_ES.UTF-8
```

Finally, export the new locale setting (or restart your terminal for the changes to take effect):

```
$ export LANG=es_ES.UTF-8
```

By ensuring your system is set up with UTF-8 encoding, you help guarantee compatibility with ROS 2 and its dependencies.

4.2. Base install

To set up the necessary environment for RMF demos, follow these steps to install the base packages and dependencies.

```
$ sudo apt install software-properties-common
$ sudo add-apt-repository universe
$ sudo apt update && sudo apt install python3-pip python3-colcon-mixin
  ros-dev-tools -y
$ sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/
  ros.key -o /usr/share/keyrings/ros-archive-keyring.gpg
$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/
  keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu
  $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo tee /etc/
  apt/sources.list.d/ros2.list > /dev/null
$ sudo apt update && sudo apt upgrade && sudo apt install ros-iron-
  desktop
$ echo "source /opt/ros/iron/setup.bash" >> ~/.bashrc # Configures ROS in
  every bash shell *optional*
$ sudo apt update && sudo apt install ros-iron-rmf-dev
```

1. Install Basic Utilities:

First, install basic utilities and add the universe repository:

```
$ sudo apt install software-properties-common
$ sudo add-apt-repository universe
```

2. Update and Install Essential Packages:

Update your package lists and install essential packages:

```
$ sudo apt update && sudo apt install python3-pip python3-colcon-
  mixin ros-dev-tools -y
```

3. Add ROS 2 Repository and Key:

Add the ROS 2 repository and its key to your system:

```
$ sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/
  master/ros.key -o /usr/share/keyrings/ros-archive-keyring.gpg
$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/
  keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2/
  ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo
  tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

4. Install ROS 2 and RMF:

Update your package lists again and install ROS 2 and RMF development tools:

```
$ sudo apt update && sudo apt upgrade
$ sudo apt install ros-iron-desktop
$ echo "source /opt/ros/iron/setup.bash" >> ~/.bashrc # Configures
    ROS in every bash shell *optional*
$ sudo apt update && sudo apt install ros-iron-rmf-dev
```

4.2.1. SocketIO Issue and Fix

During the setup, you may encounter an issue with socketio failing in the `simple_api_server`. This can be fixed by uninstalling `python3-socketio` version 5.8.0 and installing version 5.7.2. Follow these steps:

1. Uninstall Python3-SocketIO 5.8.0:

```
$ sudo pip3 uninstall python3-socketio
```

2. Install Python3-SocketIO 5.7.2:

```
$ sudo pip3 install python3-socketio==5.7.2
```

For more details, refer to the issue on GitHub: [RMF Demos Issue #177](#).

By following these steps, you will have the necessary base setup for running RMF demos and related tasks.

4.3. ROS extra configuration

ROS 2 communicates by default using a middleware called DDS, to ensure multiple ROS 2 nodes can exist in the same physical network, different logical networks are created based on the domain ID. Before starting ROS 2 it is necessary to choose a domain ID that does not interfere with existing ROS 2 instances. Furthermore, there is a soft cap on the ammount of processes you should run, as domain ID's are used to determine the UDP ports available for communication, there is a limit on the ammount of ports per domain ID which results in a total of 120 processes running. If this ammount is surpassed, there will be a conflict with the next domain ID.

To set the domain ID you need to specify the environment variable `ROS_DOMAIN_ID` with `'export ROS_DOMAIN_ID='` for a temporal change or `'echo "export ROS_DOMAIN_ID=" >> ~/.bashrc'` for a permanent change. In a linux environment the ID can be 0-101 and 215-232, other IDs could be used but this may cause collisions with other reserved ports. <https://docs.ros.org/en/iron/Concepts/Intermediate/About-Domain-ID.html#platform-specific-constraints>

When using ROS 2 the whole subnet is reachable for discovery, not only the localhost network. To prevent multiple instances of the same topics for different robots colliding in the same network like in a classroom one can set the `ROS_AUTOMATIC_DISCOVERY_RANGE` environment variable to configure the networks to discover. <https://docs.ros.org/en/iron/Tutorials/Advanced/Improved-Dynamic-Discovery.html#configuration-parameters>

- SUBNET: is the default, and for DDS based middleware it means it will discover any node reachable via multicast.
- LOCALHOST: means a node will only try to discover other nodes on the same machine.
- OFF: means the node won't discover any other nodes, even on the same machine.
- SYSTEM_DEFAULT: means "don't change any discovery settings".

If the subnet option is not available in the setup due to multiple robots in the same network, it is possible to specify distinct addresses to discover with ROS_STATIC_PEERS followed by a semicolon separated list of addresses.

```
$ export ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST
$ export ROS_STATIC_PEERS=192.168.0.1;remote.com
```

4.4. Packages Required for RMF Demo

Before setting up the RMF demos, you need to install some required packages, including Gazebo and several Python packages.

1. Install Gazebo:

First, add the Gazebo package repository and its key, then install the necessary packages:

```
$ sudo sh -c 'echo "deb http://packages.osrfoundation.org/gazebo/
  ubuntu-stable `lsb_release -cs` main" > /etc/apt/sources.list.d/
  gazebo-stable.list'
$ wget https://packages.osrfoundation.org/gazebo.key -O - | sudo apt-
  key add -
$ sudo apt-get update
$ sudo apt-get install gazebo11 libgazebo11-dev
```

2. Install Python Packages:

Next, install the required Python packages for RMF demos using pip:

```
$ python3 -m pip install flask-socketio fastapi uvicorn
```

4.5. Dependency Manager

Before installing dependencies, initialize and update rosdep. This tool helps you install system dependencies for ROS packages.

Initialize and Update rosdep:

Run the following commands to initialize and update rosdep:

```
$ sudo rosdep init # run if this is the first time using rosdep
$ rosdep update
```

4.6. Install RMF Web

To install RMF Web, follow the steps outlined below. This guide will walk you through the necessary commands to set up the environment and install the required dependencies.

```
$ curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.37.2/install.sh | bash
$ source /home/user/.bashrc
$ nvm install 16
$ curl -fsSL https://get.pnpm.io/install.sh | bash -
$ source /home/user/.bashrc
$ pip3 install pipenv
$ sudo apt install python3-venv
$ git clone https://github.com/open-rmf/rmf-web.git
$ cd rmf-web
$ pnpm install
```

1. Install NVM (Node Version Manager):

```
$ curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.37.2/install.sh | bash
```

This command downloads and runs the NVM installation script.

2. Load NVM:

```
$ source /home/takukan/.bashrc
```

Source your shell configuration file to make NVM available in the current session.

3. Install Node.js:

```
$ nvm install 16
```

Install Node.js version 16 using NVM.

4. Install pnpm:

```
$ curl -fsSL https://get.pnpm.io/install.sh | bash -
```

Download and run the installation script for pnpm, an alternative package manager for Node.js.

5. Load pnpm:

```
$ source /home/takukan/.bashrc
```

Source your shell configuration file to make pnpm available in the current session.

6. Install pipenv:

```
$ pip3 install pipenv
```

Install pipenv, a Python packaging tool that combines Pip and virtualenv.

7. Install Python Virtual Environment:

```
$ sudo apt install python3-venv
```

Install the python3-venv package, which provides support for creating lightweight virtual environments in Python.

8. Clone RMF Web Repository:

```
$ git clone https://github.com/open-rmf/rmf-web.git
```

Clone the RMF Web repository from GitHub.

9. Change to Repository Directory:

```
$ cd rmf-web
```

Navigate to the RMF Web directory.

10. Install Dependencies:

```
$ pnpm install
```

Install the required Node.js dependencies using pnpm.

By following these steps, you will have RMF Web installed and set up on your machine. This includes installing Node.js, pnpm, and pipenv, as well as cloning the RMF Web repository and installing its dependencies.

4.7. Installation check

This section provides instructions to verify the installation of ROS 2 and RMF on your system. You will learn how to run demo nodes to ensure ROS 2 is functioning correctly, by setting up a simple communication between a talker and a listener node. Additionally, we will guide you through the process of cloning and building RMF demos from source, setting up the RMF Web server, and launching a demo package. These steps will confirm that your RMF installation is correctly configured and operational.

4.7.1. ROS 2 Core

In order to check ROS 2 installation we can use demo nodes and connect a listener and a talker as shown below, the talker sends a message through a ros topic and the listener reads from that topic

```
$ ros2 run demo_nodes_cpp talker
...
[INFO] [1697541678.809331565] [talker]: Publishing: 'Hello World: 31'
[INFO] [1697541679.870864035] [talker]: Publishing: 'Hello World: 32'
[INFO] [1697541681.397645964] [talker]: Publishing: 'Hello World: 33'
[INFO] [1697541681.949364700] [talker]: Publishing: 'Hello World: 34'
[INFO] [1697541682.971568988] [talker]: Publishing: 'Hello World: 35'

$ ros2 run demo_nodes_py listener
...
[INFO] [1697541709.801463917] [listener]: I heard: [Hello World: 31]
[INFO] [1697541710.791492117] [listener]: I heard: [Hello World: 32]
[INFO] [1697541711.796455440] [listener]: I heard: [Hello World: 33]
[INFO] [1697541712.789608321] [listener]: I heard: [Hello World: 34]
[INFO] [1697541713.791782896] [listener]: I heard: [Hello World: 35]
```

4.7.2. ROS 2 RMF

The RMF (Robotics Middleware Framework) demos are not included in the standard installation, so we need to clone the repository and build them from source.

First, create a workspace and clone the RMF demos repository:

```
$ mkdir -p rmf-ws/src
$ cd rmf-ws/src
$ git clone https://github.com/open-rmf/rmf_demos.git -b 2.0.3
$ cd ..
$ colcon build
$ rosdep install --from-paths src --ignore-src --rosdistro iron -y
```

To visualize and control the simulation from the web browser, we need to run the RMF Web server before launching the demo package:

```
$ source rmf-ws/install/setup.bash
$ cd ../rmf-web/packages/dashboard
$ pnpm start
$ ros2 launch rmf_demos_gz office.launch.xml server_uri:="ws://localhost:8000/_internal"
```

1. Create Workspace and Clone Repository:

```
$ mkdir -p rmf-ws/src
$ cd rmf-ws/src
$ git clone https://github.com/open-rmf/rmf_demos.git -b 2.0.3
```

This creates a new workspace directory, navigates into it, and clones the RMF demos repository from GitHub.

2. Build the Workspace:

```
$ cd ..
$ colcon build
```

Navigate back to the workspace root and build the source code using colcon.

3. Install Dependencies:

```
$ rosdep install --from-paths src --ignore-src --rosdistro iron -y
```

Install any dependencies required by the RMF demos using rosdep.

4. Source the Workspace Setup:

```
$ source rmf-ws/install/setup.bash
```

Source the workspace setup file to overlay the new packages into your environment.

5. Start RMF Web Dashboard:

```
$ cd ../rmf-web/packages/dashboard
$ pnpm start
```

Navigate to the RMF Web dashboard package and start the web server using pnpm.

6. Launch the RMF Demo:

```
$ ros2 launch rmf_demos_gz office.launch.xml server_uri:="ws://localhost:8000/_internal"
```

Launch the RMF demo, specifying the web server URI to enable web-based visualization and control.

By following these steps, you will be able to set up, build, and run the RMF demos, as well as visualize and control the simulation from a web browser using the RMF Web dashboard.

5. First project

This section will guide you through creating your first RMF project using the instruction from the ROS2 Multirobot book - Traffic editor. We will follow the project structure of RMF demos, setting up necessary packages, and configuring dependencies and project settings in package.xml and CMakeLists.txt files. You will learn how to create a scenario configuration, define launch parameters, and set up fleet and robot configurations. Additionally, we will cover the setup of simulation launch files for testing your project in a simulated environment. By the end of this section, you will have a fully configured and functional RMF project ready for development and testing.

5.1. Create Project Structure and Basic Configuration

We will follow the project structure of RMF demos but will ignore some of the packages for now.

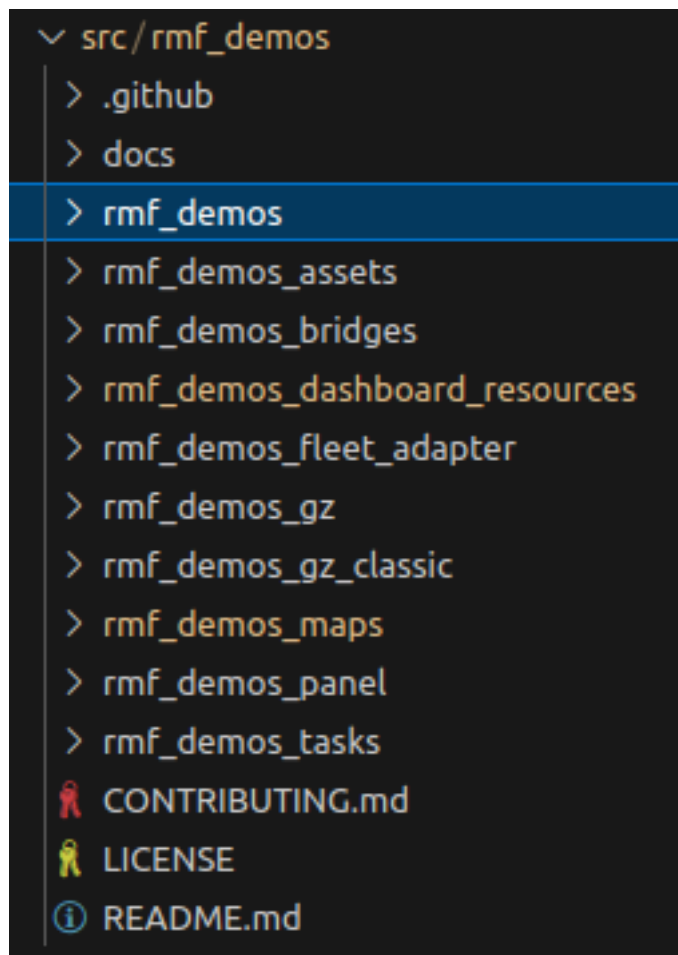


Figure 1: Project Structure

5.1.1. Copy and Create the Following Packages

1. Navigate to the rmf-ws/src/ Directory and Create Your Project Directory:

```
$ cd rmf-ws/src/
```

```
$ mkdir project # change 'project' to your project name, we will use
                  'project' from now on
$ cd project
```

This creates a directory named `project` (replace `project` with your actual project name).

2. Copy the Necessary Assets and Create New ROS 2 Packages:

```
$ cp -r ../rmf_demos/rmf_demos_assets project_assets
$ ros2 pkg create project_config --license Apache-2.0
$ cp -r ../rmf_demos/rmf_demos_fleet_adapter project_fleet_adapter
$ ros2 pkg create project_maps --license Apache-2.0
$ ros2 pkg create project_simulation --license Apache-2.0
```

- `cp -r ../rmf_demos/rmf_demos_assets project_assets`: Copies the `rmf_demos_assets` directory from the RMF demos into your project, renaming it to `project_assets`.
- `ros2 pkg create project_config --license Apache-2.0`: Creates a new ROS 2 package named `project_config` with the specified license.
- `cp -r ../rmf_demos/rmf_demos_fleet_adapter project_fleet_adapter`: Copies the `rmf_demos_fleet_adapter` directory from the RMF demos into your project, renaming it to `project_fleet_adapter`.
- `ros2 pkg create project_maps --license Apache-2.0`: Creates a new ROS 2 package named `project_maps` with the specified license.
- `ros2 pkg create project_simulation --license Apache-2.0`: Creates a new ROS 2 package named `project_simulation` with the specified license.

3. Change the Name from `rmf_demos` to Your Project Name:

```
$ cd rmf-ws/src/project
$ find . -name 'rmf_demos*' -print -exec bash -c 'mv $0 ${0/rmf_demos
  /project}' {} \;
$ find . -name 'rmf_demos*' -print -exec bash -c 'mv $0 ${0/rmf_demos
  /project}' {} \;
$ find . -exec sed -i -e 's/rmf_demos/project/g' {} \;
```

- The first `find` command searches for files and directories named `rmf_demos*` and renames them to `project*`.
- The second `find` command repeats the renaming process to ensure all subfolders are correctly renamed.
- The third `find` command searches all files and replaces instances of `rmf_demos` with `project`.

5.1.2. Set Up Dependencies and Project Configuration

Next, configure the dependencies and project settings in each project's corresponding `package.xml` and `CMakeLists.txt` files.

The `package.xml` file contains information about the package such as name, version, maintainer, etc. It is important to pay attention to the `buildtool_depend` and `exec_depend` fields, where the former specifies the tooling required to build the project, while the latter specifies the dependencies of the package.

1. Edit `project_config/package.xml`:

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
  schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>project_config </name>
  <version>0.0.0</version>
  <description>Common launch files </description>
  <maintainer email="mail">email</maintainer>
  <license>Apache License 2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <exec_depend>project_maps </exec_depend>
  <exec_depend>rmf_traffic_ros2 </exec_depend>
  <exec_depend>rmf_task_ros2 </exec_depend>
  <exec_depend>rmf_fleet_adapter </exec_depend>
  <exec_depend>rmf_building_map_tools </exec_depend>
  <exec_depend>rmf_visualization </exec_depend>
  <exec_depend>project_assets </exec_depend>
  <exec_depend>project_tasks </exec_depend>
  <exec_depend>project_panel </exec_depend>
  <exec_depend>project_fleet_adapter </exec_depend>
  <exec_depend>rviz2 </exec_depend>
  <exec_depend>launch_xml </exec_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

- `<name>project_config</name>`: Specifies the name of the package.

- `<version>0.0.0</version>`: Specifies the version of the package.
- `<maintainer email="mail">email</maintainer>`: Specifies the maintainer's email.
- `<buildtool_depend>ament_cmake</buildtool_depend>`: Specifies `ament_cmake` as a build tool dependency.
- `<exec_depend>...`: Specifies the runtime dependencies for the package.

The `CMakeLists.txt` file contains the build instructions for the package.

1. Edit `project_config/CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.8)
project( project_config )

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# find dependencies
find_package(ament_cmake REQUIRED)
# uncomment the following section in order to fill in
# further dependencies manually.
# find_package(<dependency> REQUIRED)

install(DIRECTORY
  launch /
  config /
  DESTINATION share/${PROJECT_NAME}
)

ament_package()
```

- `cmake_minimum_required(VERSION 3.8)`: Sets the minimum required version of CMake.
- `project( project_config )`: Sets the project name.
- `if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")...`: Adds compiler options for GCC or Clang.
- `find_package(ament_cmake REQUIRED)`: Finds the `ament_cmake` package which is required for building.
- `install(DIRECTORY launch/ config/ DESTINATION share/${PROJECT_NAME})`: Installs the `launch` and `config` directories to the specified destination.
- `ament_package()`: This macro must be called for any package to be recognized by ROS 2's build system.

By following these steps, you will set up the basic structure and configuration for your project based on the RMF demos. Ensure that you adjust the `package.xml` and `CMakeLists.txt` files according to your project's specific requirements and dependencies.

5.2. Configure scenario

To start using the Robotics Middleware Framework (RMF), you need to create a scenario, which involves setting up the configuration and launch files in `ros2` launch file formats necessary for the RMF to operate. This guide will help you create your first scenario, named 'test1'.

RMF configuration is extensive and complex, it may take some time to understand what anything does. The first section of these files are the arguments, if no value is set and no default value is provided it throws an error. The other part works by composing different `launch.xml` files that reference to one another, this is where you can set the values of the arguments, think of them as calls to other modules.

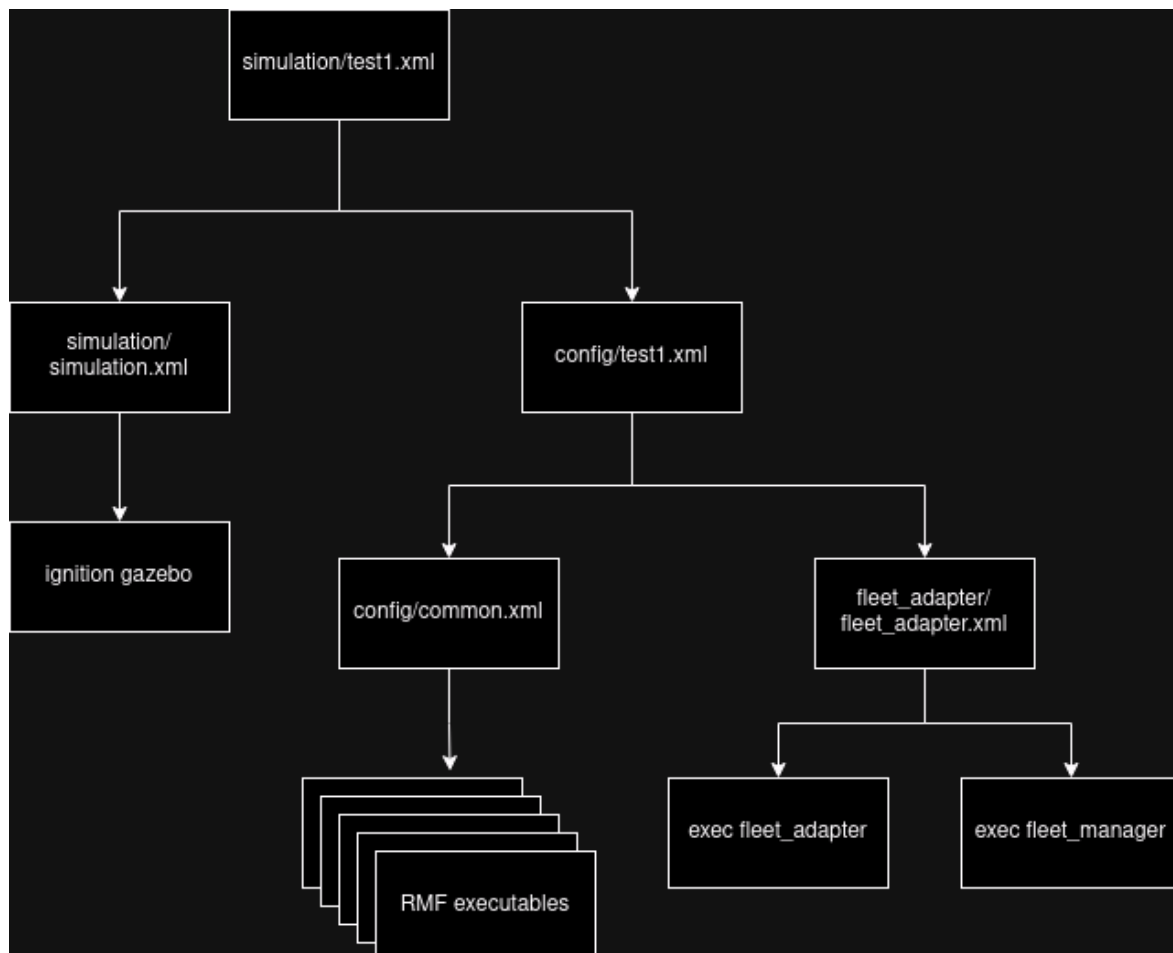


Figure 2: Launch files composition structure

In the configuration package (`project_config`), you need to add two folders: `config` and `launch`. Similarly, in

the simulation package (project_simulation), you need to add a launch folder. These folders will contain the necessary files to set up and run the scenario.

5.2.1. Project config

project_config/launch/common.launch.xml

This file launches the main functionality of RMF, such as the traffic scheduler, supervisors, and dispatchers. Below is an explanation of its structure:

```
<?xml version = '1.0' ?>

<launch>

  <arg name="use_sim_time" default="false" description="Use the /clock
    topic for time to sync with simulation"/>
  <arg name="viz_config_file" default="$(find -pkg-share
    rmf_visualization_schedule)/config/rmf.rviz"/>
  <arg name="config_file" description="Building description file required
    by rmf_building_map_tools"/>
  <arg name="dashboard_config_file" default="" description="Path to
    dashboard config for web rmf panel file"/>
  <arg name="initial_map" default="L1" description="Initial map name for
    the visualizer"/>
  <arg name="headless" default="false" description="do not launch rviz;
    launch gazebo in headless mode"/>
  <arg name="bidding_time_window" description="Time window in seconds for
    task bidding process" default="2.0"/>
  <arg name="use_rmf_panel" default="true" description="launch web and
    api server for rmf_demos_panel"/>

  <!-- Traffic Schedule -->
  <node pkg="rmf_traffic_ros2" exec="rmf_traffic_schedule" output="both"
    name="rmf_traffic_schedule_primary">
    <param name="use_sim_time" value="$(var use_sim_time)"/>
  </node>

  <!-- Blockade Moderator -->
  <node pkg="rmf_traffic_ros2" exec="rmf_traffic_blockade" output="both">
    <param name="use_sim_time" value="$(var use_sim_time)"/>
  </node>
```

```

<!-- Building Map -->
<group>
  <node pkg="rmf_building_map_tools" exec="building_map_server" args="$
    (var config_file)">
    <param name="use_sim_time" value="$(var use_sim_time)"/>
  </node>
</group>

<!-- Visualizer -->
<group>
  <include file="$(find-rmf-share rmf_visualization)/visualization.
    launch.xml">
    <arg name="use_sim_time" value="$(var use_sim_time)"/>
    <arg name="map_name" value="$(var initial_map)"/>
    <arg name="viz_config_file" value="$(var viz_config_file)"/>
    <arg name="headless" value="$(var headless)"/>
  </include>
</group>

<!-- Door Supervisor -->
<group>
  <node pkg="rmf_fleet_adapter" exec="door_supervisor">
    <param name="use_sim_time" value="$(var use_sim_time)"/>
  </node>
</group>

<!-- Lift Supervisor -->
<group>
  <node pkg="rmf_fleet_adapter" exec="lift_supervisor">
    <param name="use_sim_time" value="$(var use_sim_time)"/>
  </node>
</group>

<!-- Dispatcher Node -->
<group>
  <node pkg="rmf_task_ros2" exec="rmf_task_dispatcher" output="screen
    ">
    <param name="use_sim_time" value="$(var use_sim_time)"/>

```

```

        <param name="bidding_time_window" value="$(var bidding_time_window)
        "/>
    </node>
</group>

<!-- <group if="$(var use_rmf_panel)">
    <include file="$(find-pkg-share rmf_demos)/dashboard.launch.xml">
        <arg name="use_sim_time" value="$(var use_sim_time)"/>
        <arg name="dashboard_config_file" value ="$(var
            dashboard_config_file)"/>
    </include>
</group> -->

</launch>

```

Explanation:

- **Arguments (<arg>):** These are configurable parameters for the launch file. For instance, `use_sim_time` determines if the simulation time is used and it is a very importante parameter, when start the simulation you decide to use sim time or not, any other execution to that simulation like a task or a new package must also use the same option in `use_sim_time` or it won't work.
- **Nodes (<node>):** These represent individual processes that are launched. Each node belongs to a package and executes a specific program. There some from RMF like traffic scheduler and blockade moderator that control the pathing system, others like building map tools that are used to parse the yaml file created with `traffic_editor` to a rmf-compatible file and other tools like supervisors to make door and lifts work or visualizer that launches rviz.
- **Groups (<group>):** These are used to logically group nodes and other launch configurations together.

project_config/launch/test1.launch.xml

This file defines scenario-specific launch parameters and executes the common launch. It essentially includes the common launch file and adds specific configurations for the 'test1' scenario.

```

<?xml version='1.0' ?>

<launch>
    <arg name="use_sim_time" default="false"/>

    <!-- Common launch -->
    <include file="$(find-pkg-share project_config)/common.launch.xml">
        <arg name="use_sim_time" value="$(var use_sim_time)"/>
        <arg name="viz_config_file" value ="$(find-pkg-share project_config)/

```

```

    include / test1 / test1 . rviz "/>
    <arg name="config_file" value="$(find -pkg-share project_maps) / test1 /
    test1 . building . yaml"/>
    <!-- <arg name="dashboard_config_file" value="$(find -pkg-share
    config_dashboard_resources) / test1 / dashboard_config . json"/> -->
</include>

<!-- Tinyrobot fleet adapter -->
<group>
    <include file="$(find -pkg-share project_fleet_adapter) / launch /
    fleet_adapter . launch . xml">
    <arg name="use_sim_time" value="$(var use_sim_time)"/>
    <arg name="nav_graph_file" value="$(find -pkg-share project_maps) /
    maps / test1 / nav_graphs / 1 . yaml" />
    <arg name="config_file" value="$(find -pkg-share project_config) /
    config / test1 / tinyRobot_config . yaml"/>
    </include>
</group>

</launch>

```

Explanation:

- This file uses the `<include>` tag to incorporate other launch files, effectively creating a modular launch setup that cascades into launching the whole system.
- Arguments specific to the scenario 'test1' are set here and passed onto the other launch files.
- Take special attention to 'nav_graph_file', this are the lanes the the robot can traverse in the map. It will be explained latter in the create map section.

project_config/config/test1/tinyRobot_config.yaml

This YAML file defines the configuration for the tiny robot fleet. It includes both fleet-level and robot-specific parameters:

```

# FLEET CONFIG
=====
# RMF Fleet parameters

rmf_fleet:
  name: "tinyRobot"
  fleet_manager:
    ip: "127.0.0.1"

```

```

    port: 22011
    user: "some_user"
    password: "some_password"
limits:
    linear: [0.5, 0.75] # velocity, acceleration
    angular: [0.6, 2.0] # velocity, acceleration
profile: # Robot profile is modelled as a circle
    footprint: 0.3 # radius in m
    vicinity: 0.5 # radius in m
reversible: True # whether robots in this fleet can reverse
battery_system:
    voltage: 12.0 # V
    capacity: 24.0 # Ahr
    charging_current: 5.0 # A
mechanical_system:
    mass: 20.0 # kg
    moment_of_inertia: 10.0 #kgm^2
    friction_coefficient: 0.22
ambient_system:
    power: 20.0 # W
tool_system:
    power: 0.0 # W
recharge_threshold: 0.10 # Battery level below which robots in this
    fleet will not operate
recharge_soc: 1.0 # Battery level to which robots in this fleet should
    be charged up to during recharging tasks
publish_fleet_state: 10.0 # Publish frequency for fleet state, ensure
    that it is same as robot_state_update_frequency
account_for_battery_drain: True
task_capabilities: # Specify the types of RMF Tasks that robots in this
    fleet are capable of performing
loop: True
delivery: True
clean: False
finishing_request: "park" # [park, charge, nothing]

# TinyRobot CONFIG
=====

```

```

robots :
  # Here the user is expected to append the configuration for each robot
  # in the
  # fleet .
  # Configuration for tinyRobot1
  tinyRobot1 :
    robot_config :
      max_delay: 15.0 # allowed seconds of delay of the current itinerary
      before it gets interrupted and replanned
    rmf_config :
      robot_state_update_frequency: 10.0
      start :
        map_name: "L1"
        waypoint: "tinyRobot1_charger"
        orientation: 0.0 # radians
      charger :
        waypoint: "tinyRobot1_charger"

```

Explanation:

- **Fleet Configuration (rmf_fleet):** Contains overall parameters for the fleet, such as server configuration, speed limits, battery system specifications, and task capabilities.
- **Robot Configuration (robots):** Details specific to each robot in the fleet, including their initial state and charging station.

5.2.2. Project simulation

project_simulation/launch/simulation.launch.xml

This file launches the Gazebo simulator, setting up the simulation environment:

```

<?xml version='1.0' ?>

<launch>
  <arg name="map_package" default="project_maps" description="Name of the
    map package" />
  <arg name="map_name" description="Name of the project map to simulate"
    />
  <arg name="use_crowdsim" default='0' />
  <arg name="use_tpe" default='0' />
  <arg name="ignition_version" default='6' />

```

```

<let name="world_path" value="$(find-pkg-share $(var map_package))/maps
    /$(var map_name)_ign/$(var map_name).world" />
<let name="model_path" value="$(find-pkg-share $(var map_package))/maps
    /$(var map_name)/models:$(find-pkg-share $(var map_package))/maps/$(
    var map_name)_ign/models:$(find-pkg-share project_assets)/models:$(
    env HOME)/.gazebo/models" />
<let name="plugin_path" value="$(find-pkg-prefix
    rmf_robot_sim_gz_plugins)/lib/rmf_robot_sim_gz_plugins:$(find-pkg-
    prefix rmf_building_sim_gz_plugins)/lib/rmf_building_sim_gz_plugins"
    />
<!-- Use TPE if `use_tpe` is set to true-->
<!-- TODO use the variable below and remove code duplication (second
    ign gazebo command) when substitution in XML with ignition gazebo
    works -->
<let name="tpe_engine" value="\-physics-engine ignition-physics-tpe-
    plugin" />
<let name="tpe_engine" unless="$(var use_tpe)" value="" />

<!-- Use crowd sim if `use_crowdsim` is true-->
<let name="menge_resource_path" if="$(var use_crowdsim)" value="$(find-
    pkg-share $(var map_package))/maps/$(var map_name)/config_resource
    "/>
<let name="menge_resource_path" unless="$(var use_crowdsim)" value=""
    />

<let name="ign_headless" if="$(var headless)" value="-s"/>
<let name="ign_headless" unless="$(var headless)" value="" />

<group unless="$(var use_tpe)">
    <executable cmd="ign gazebo --force-version $(var ignition_version) $
        (var ign_headless) $(var tpe_engine) -r -v 4 $(var world_path)"
        output="both">
        <env name="IGN_GAZEBO_RESOURCE_PATH" value="$(var model_path):$(var
            world_path)" />
        <env name="IGN_GAZEBO_SYSTEM_PLUGIN_PATH" value="$(var plugin_path)
            "/>
        <env name="IGN_GUI_PLUGIN_PATH" value="$(var plugin_path)"/>
        <env name="MENGE_RESOURCE_PATH" value="$(var menge_resource_path)
            "/>

```



```

    </executable>
</group>

<group if ="$(var use_tpe)">
  <executable cmd="ign gazebo $(var ign_headless) --physics-engine
    ignition-physics-tpe-plugin -r -v 4 $(var world_path)" output="
    both">
    <env name="IGN_GAZEBO_RESOURCE_PATH" value="$(var model_path):$(var
      world_path)" />
    <env name="IGN_GAZEBO_SYSTEM_PLUGIN_PATH" value="$(var plugin_path)
      "/>
    <env name="IGN_GUI_PLUGIN_PATH" value="$(var plugin_path)"/>
    <env name="MENGE_RESOURCE_PATH" value="$(var menge_resource_path)
      "/>
  </executable>
</group>

<!-- ros_ign bridge for simulation clock -->
<node pkg="ros_ign_bridge" exec="parameter_bridge"
  args="/clock@rosgraph_msgs/msg/Clock[ignition.msgs.Clock]"
  />

</launch>

```

Explanation:

- **Arguments (<arg>):** These set up various parameters for the simulation, such as which map to use(map_name), where to find it(map_package) and whether to use crowd simulation.
- **Variable (<let>):** These create different variables for the simulation and there are defined based on the arguments.
- **Executable (<executable>):** This runs the Gazebo simulation, with different configurations based on whether TPE (Time Parameterized Execution) is used.
- **Node (<node>):** This sets up the bridge between ROS and Gazebo, allowing ROS to control the Gazebo simulation.

project_simulation/launch/test1.launch.xml

This file is the main entry point, invoking the necessary launch files to run the ‘test1’ scenario:

```

<?xml version='1.0' ?>

<launch>

```

```

<arg name="use_sim_time" default="true"/>
<arg name="failover_mode" default="false"/>
<arg name="use_tpe" default="false"/>
<arg name="ignition_version" default='6'/>

<!-- Common launch -->
<include file="$(find-pkg-share project_config)/test1.launch.xml">
  <arg name="use_sim_time" value="$(var use_sim_time)"/>
  <arg name="failover_mode" value="$(var failover_mode)"/>
  <arg name="use_tpe" value="$(var use_tpe)"/>
</include>

<!-- Simulation launch -->
<include file="$(find-pkg-share project_simulation)/simulation.launch.xml">
  <arg name="map_name" value="test1" />
  <arg name="ignition_version" value="$(var ignition_version)" />
</include>

</launch>

```

Explanation:

- This file combines the common launch and simulation launch files to set up and run the ‘test1’ scenario.
- Arguments specific to the ‘test1’ scenario are set here, such as use_sim_time and failover_mode.

5.3. Create a Map

To create a map, we can use the ‘traffic-editor’ tool that comes with RMF, we will guide you through the process step by step with examples and you can also check the ROS2 traffic editor documentation. Execute `traffic – editor` in the terminal, go to Building → New, and create a new `test1 . building .yaml` file in `project_maps/maps/test1 /`. Select ‘reference image coordinates’.

1. Create a Level:

- Press the “Add...” button on the top-right side of the screen. Name the level L1 and press OK as shown in the Figure 3.

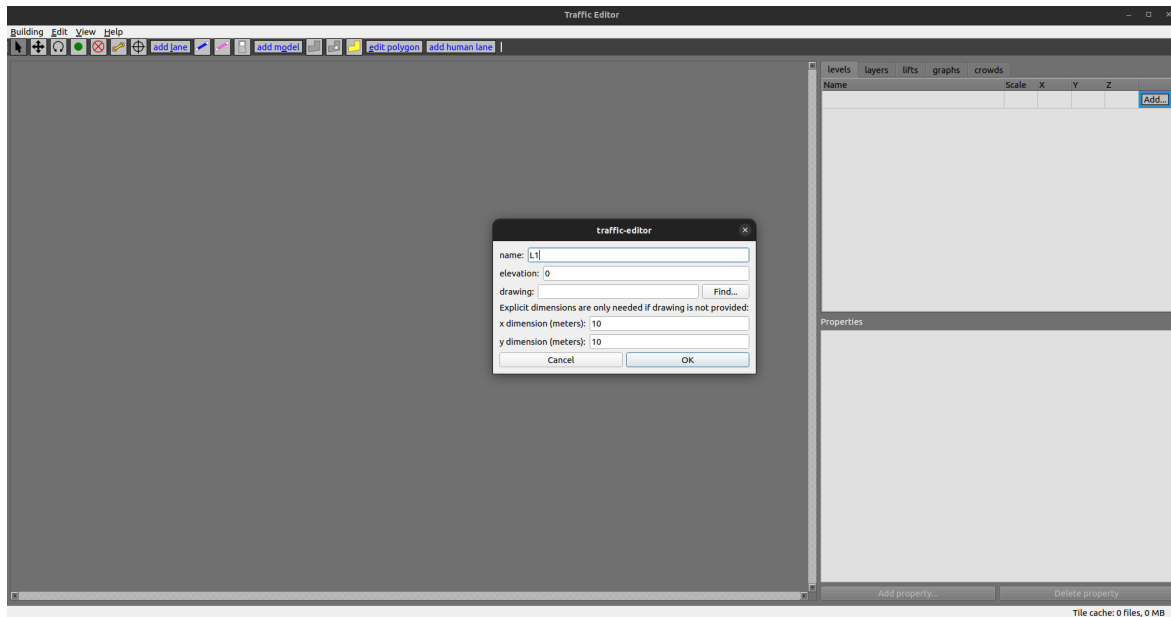


Figure 3: Creating a New Level

2. Add Walls:

- Press the wall button (marked as a blue line) or press 'W' to use the add wall tool. The Wall tool will add vertices for you and join them with walls (Figure 4). You can also set the vertices (green dot or press 'V') before the walls for more control over the layout. Vertices are key for the level layout as they can become walls, doors, floors, measurements, lanes, and more.

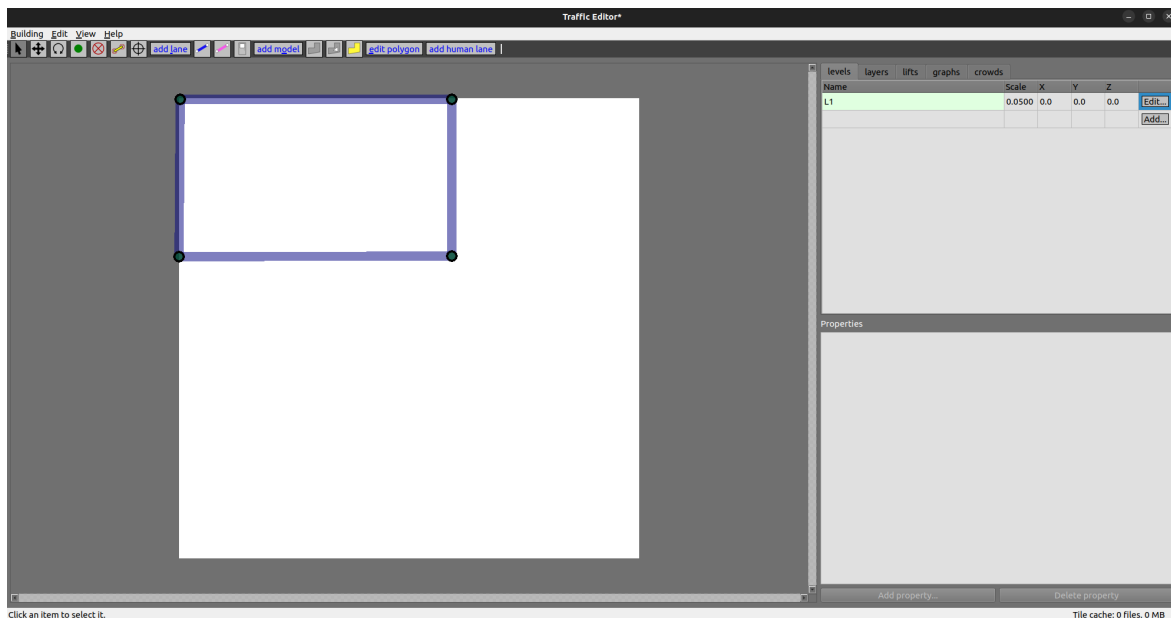


Figure 4: Adding Walls

3. Align Vertices:

- Since this process is manual there is always error in the placement so we need to fix the position of the vertices manually. Select the top-left vertex using the select tool (cursor button or press 'Esc') and check its properties on the bottom-right panel. You will notice the 'x (pixels)' and 'y (pixels)' values. Set both to 0 and align the other three vertices to create a rectangle with a size of 100x150 as reflected in Figure 5.

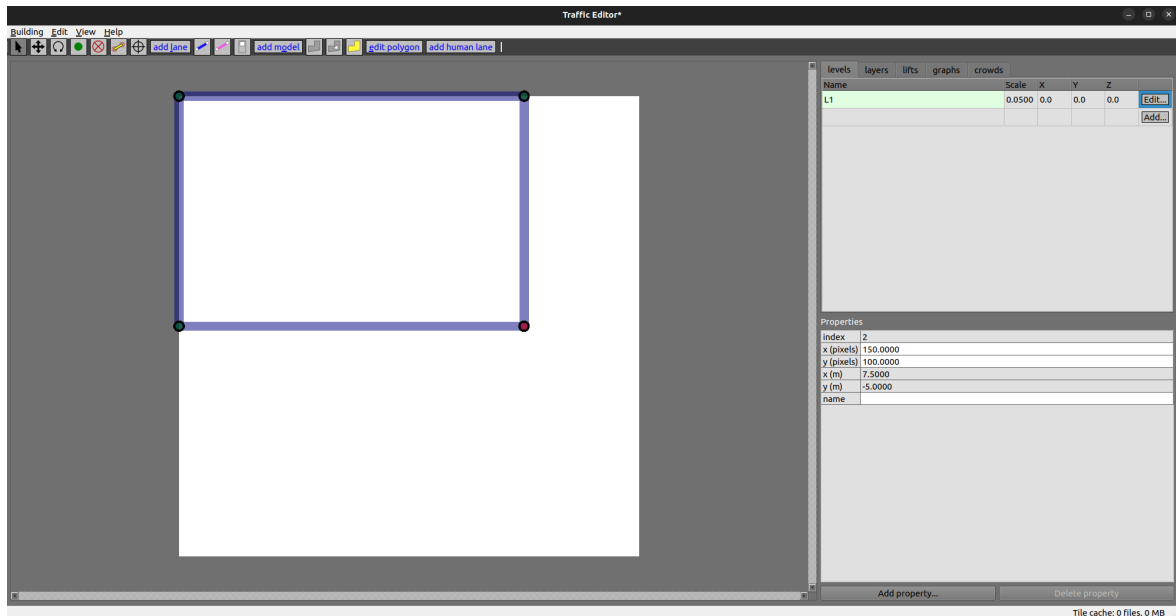


Figure 5: Aligning Walls

4. Add a Floor:

- Use the same vertices to add the floor with the floor tool (marked as a grey polygon or press 'F'), just click all the vertices in clock-wise or counter clock-wise order and right-click when done(Figure 6).

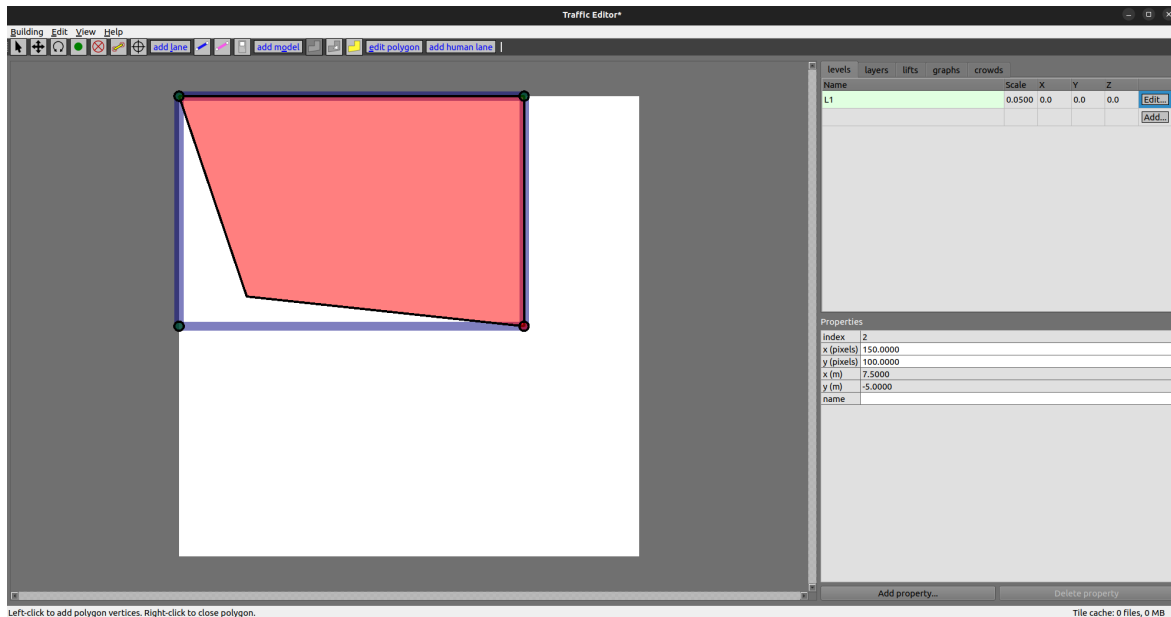


Figure 6: Adding a Floor

5. Spawn a Robot:

- Create a vertex at coordinates (50, 50), remember, use 'V' to apply the vertex tool. Set the x and y properties to 50 and add the spawn_robot_name, spawn_robot_type, and is_charger properties (Figure 7), this is done by clicking in the "Add property..." button and selection the proper options in the dropdown menu. Set the name of the vertex to tinyRobot1_charger, is_charger to true, spawn_robot_name to tinyRobot1 and spawn_robot_type to TinyRobot (Figure 8). Check the configuration file project_config / config / test1 / tinyRobot_config .yaml as this properties should have the same names.

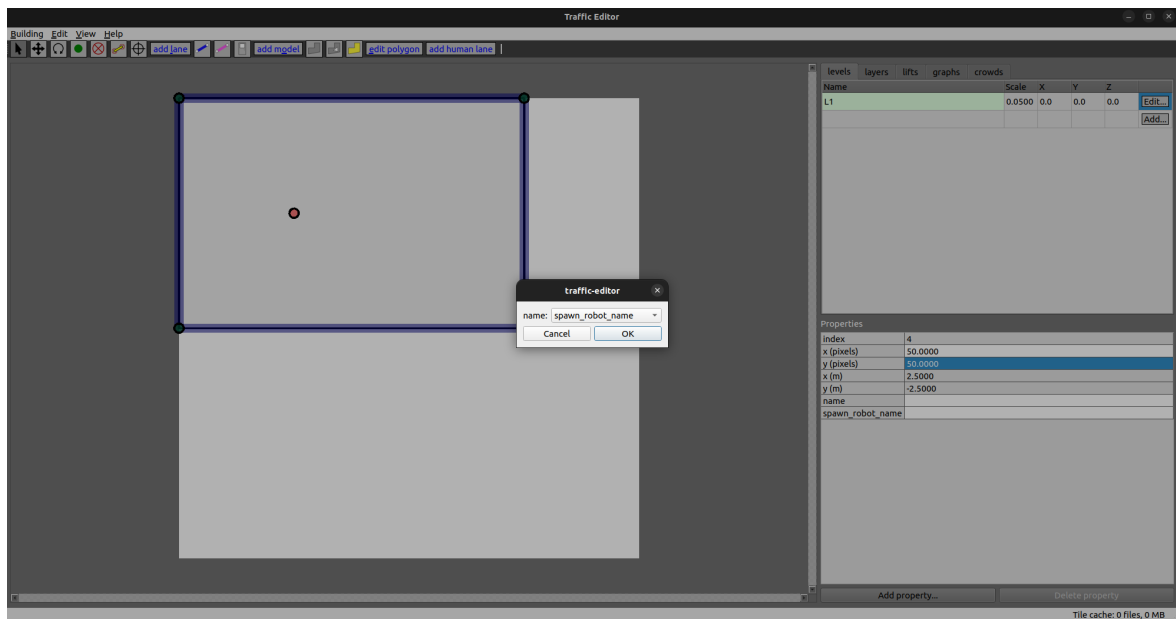


Figure 7: Spawning a Robot

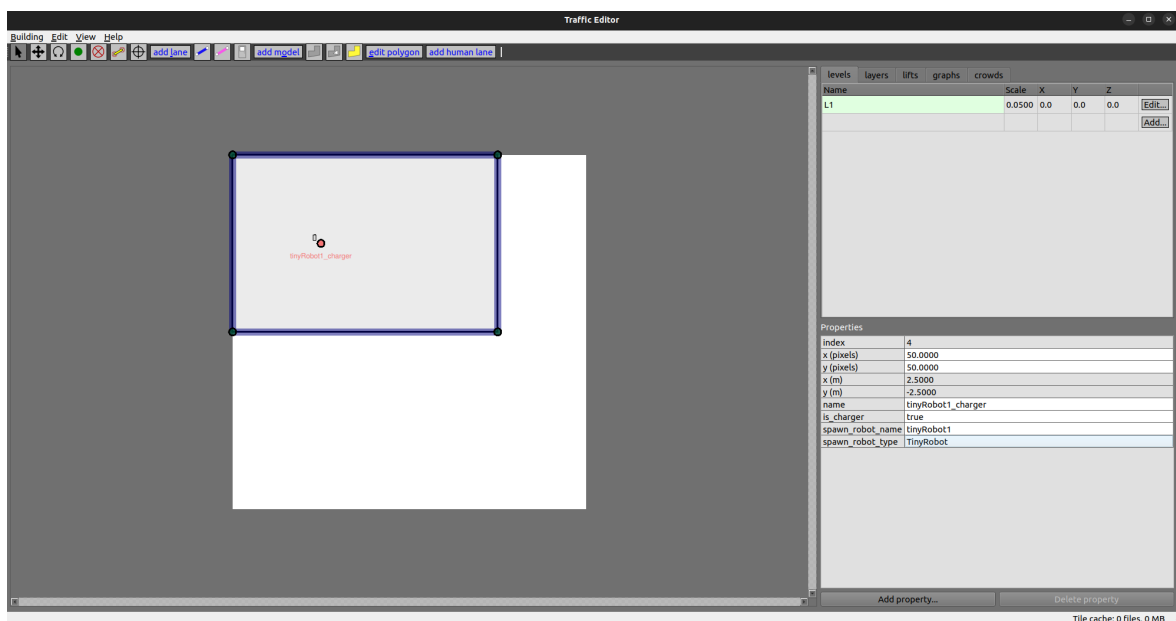


Figure 8: Robot Configuration

6. Add a Lane:

- To allow the robot to move, click on add_lane or press 'L' to invoke the lane tool. Select the vertex used for the robot spawn point and create a lane by clicking on another vertex or in any place in the map (this will create a new vertex). Press 'Esc' and if you created a new vertex select it, then change its x and y coordinates to (100, 50) and name it waypoint1 (Figure 9).

- It is possible to create different lane levels for different robots, by pressing the number from 0 to 9, this will assign a different color to the lanes you create. In this example we are focusing on just one lane, usually '0' by default, but in this case we will use '1' by pressing the number '1'. If you want to use the different navigation graphs that are created from this, you will need to add in each robot's yaml configuration the proper `nav_graph_file` property.

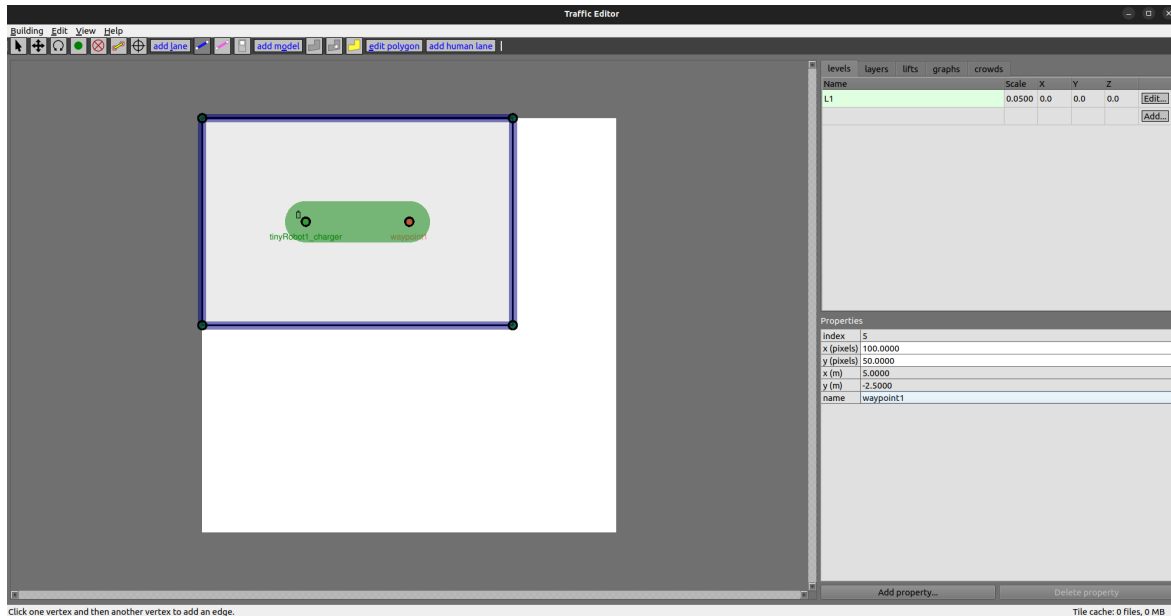


Figure 9: Adding a Lane

7. Run the Simulation:

- We configured a small room with a robot, let's check in the simulation. Use `colcon` to build the project, source the `setup.bash`, and launch the simulation with the following command, this will build the project and set up all dependencies for ROS to launch the simulation:

```
colcon build && source install/setup.bash && ros2 launch
  project_simulation test1.launch.xml
```

- As observed in Figure 10, two windows should pop up: Gazebo (with a 3D representation of the map) and RViz (initially empty but can be configured to show important information about lanes and robots).

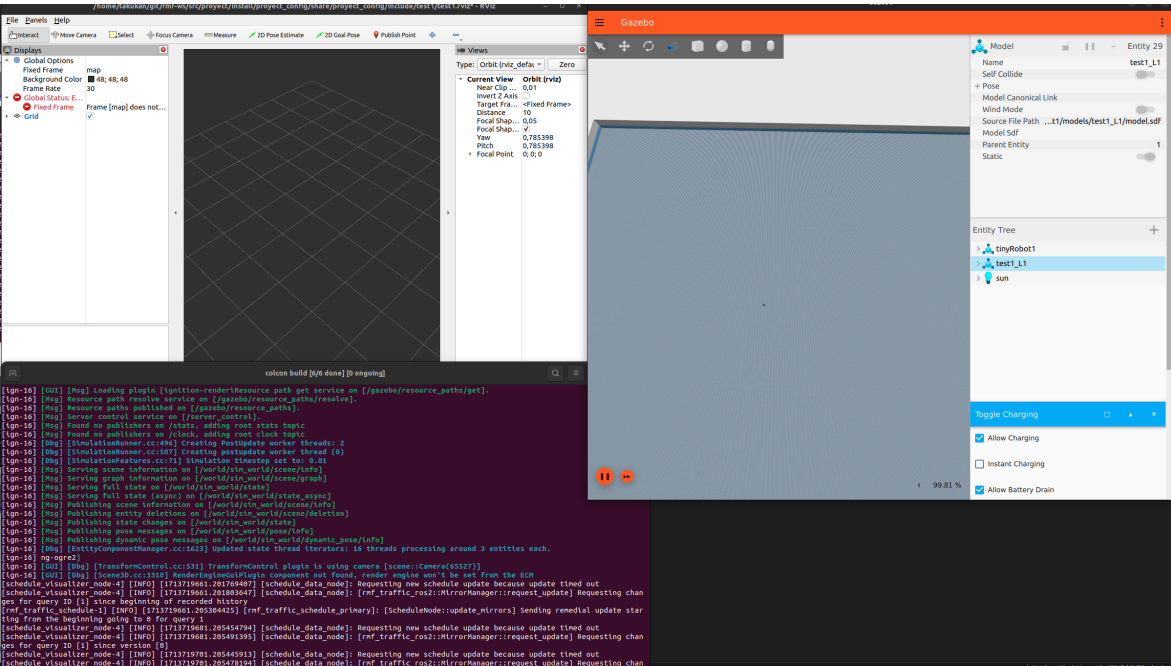


Figure 10: First Launch

8. Set Measurements:

- The robot appears tiny because no measurements have been set so the system uses one by default. Click on the pink line next to the wall or press 'T' to use the measurement tool. Click on two vertices and set the distance between them. Click on the measurement line and adjust the distance to a more comfortable size (e.g., 10 units). Usually, the layout is created using a blueprint of the building with proper measures.

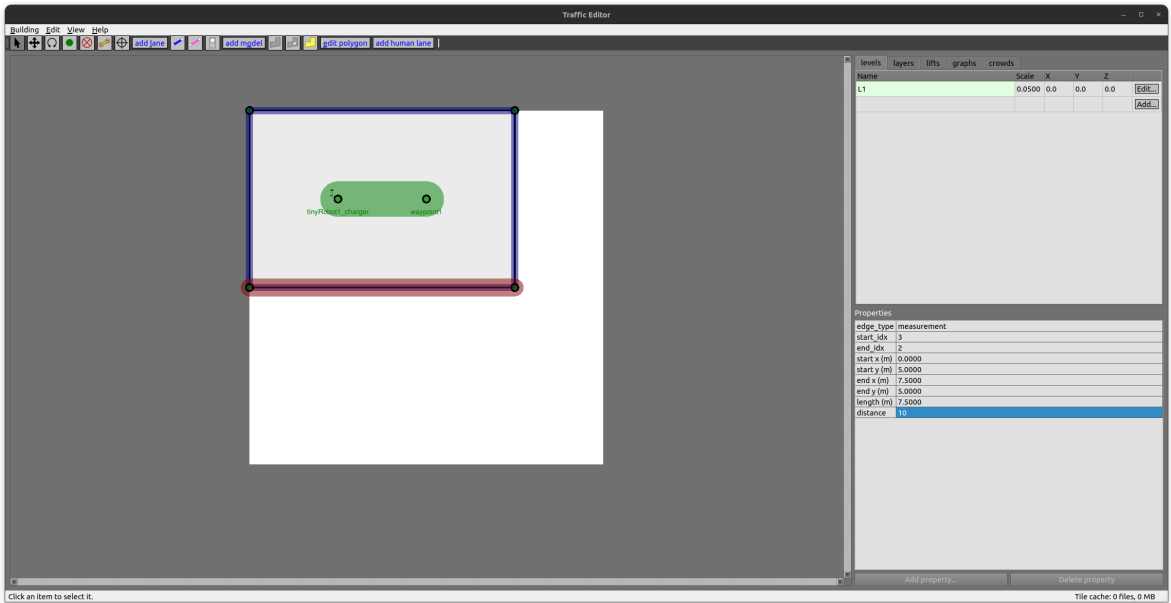


Figure 11: Setting Measurements

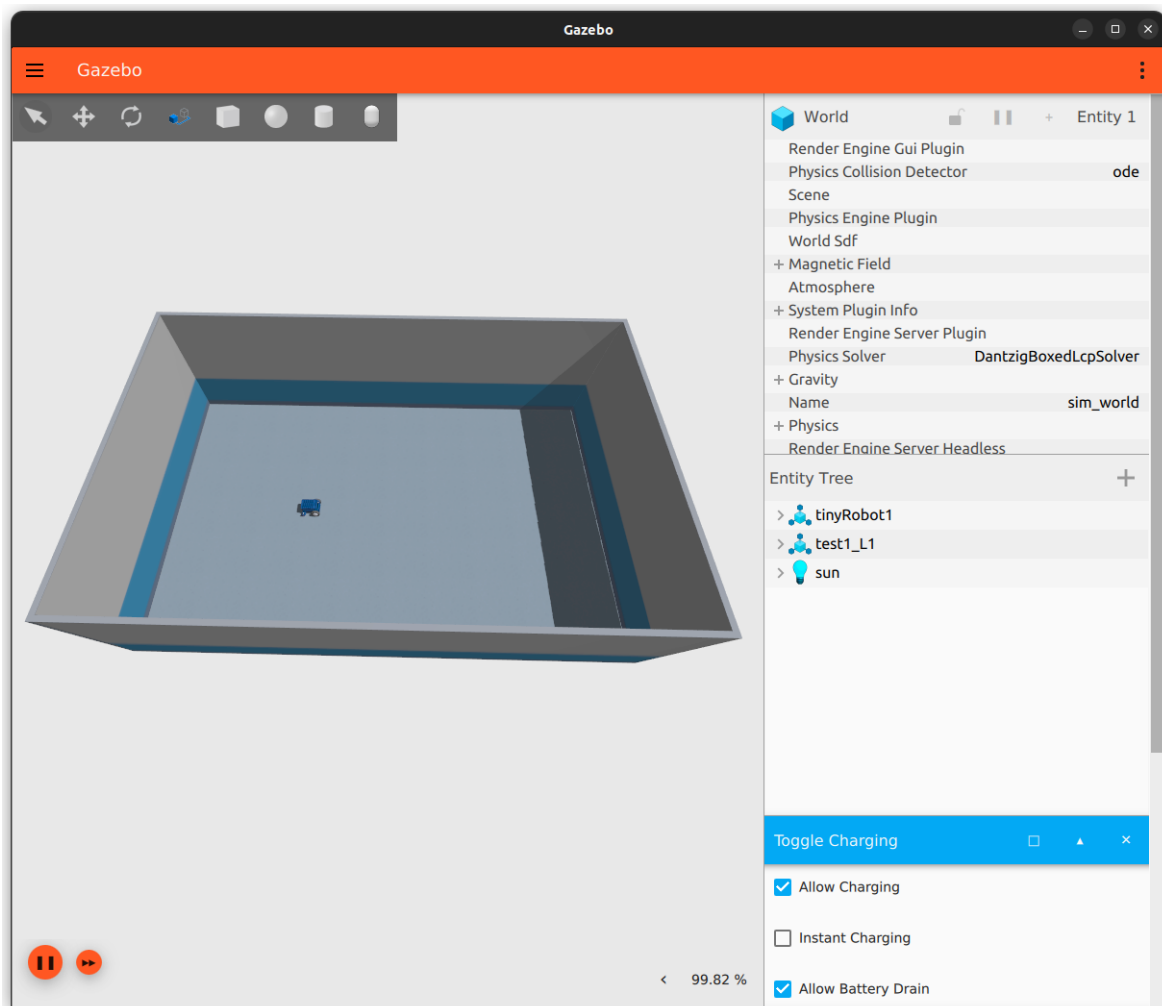


Figure 12: Measurement Simulation

9. Add Doors and Walls:

- Add a door using the door button (or press 'D') and create both ends of the door, the order in which you add the two points for the door decides its orientation but that can be changed modifying the “motion direction” property of the door to -1. Remember to always set a name to your doors, if there is more than one door with an empty name the simulation will crash.

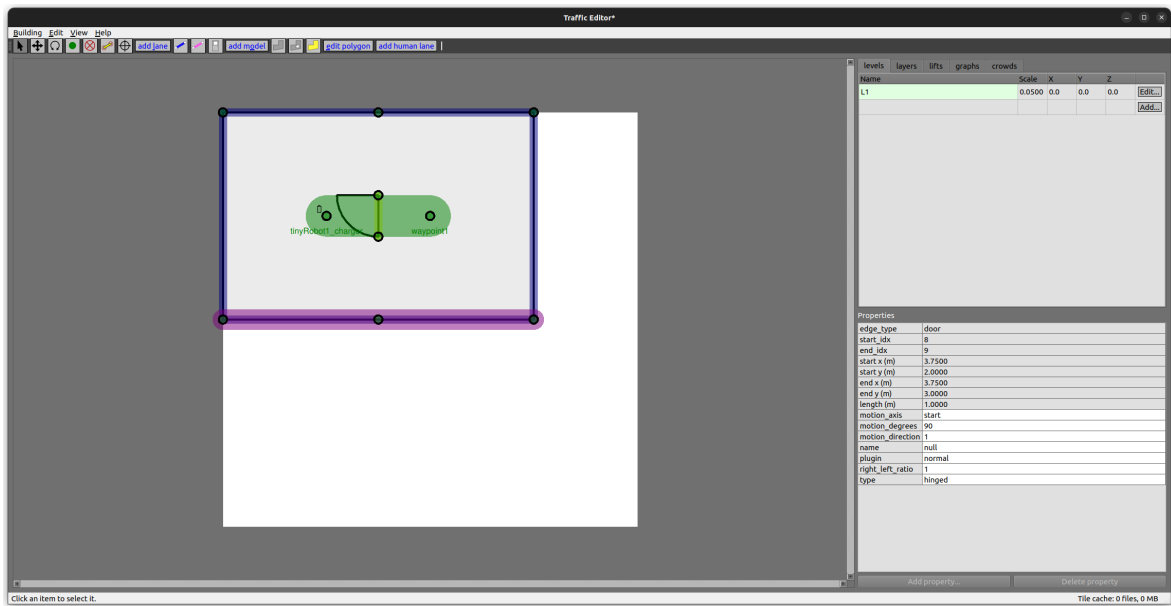


Figure 13: Adding a Door

- Add walls to create two rooms that can be traversed through the door to create a similar scenario to Figure 12.

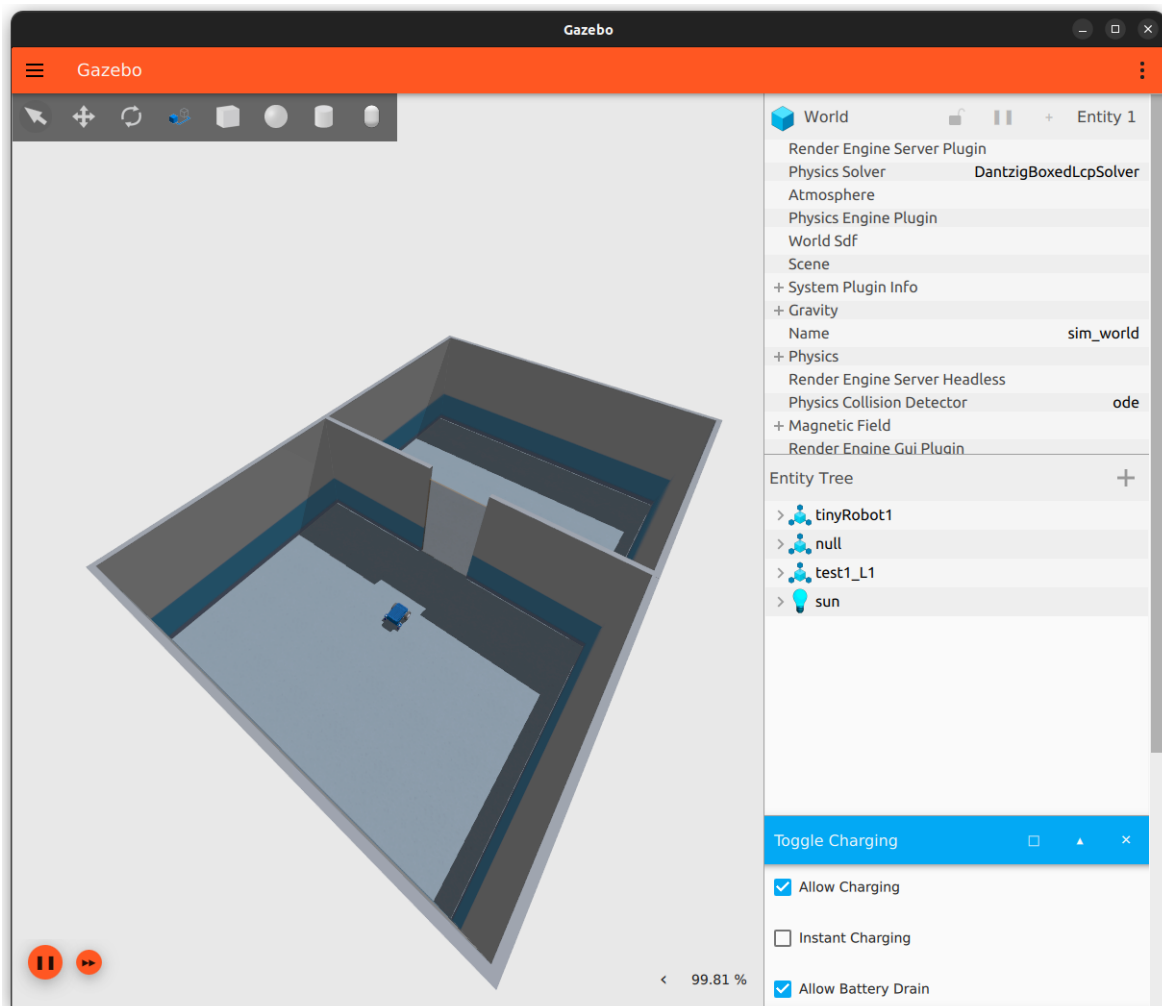


Figure 14: Door Simulation

- Now you can send a task to the robot to move to the waypoint1 with:

```
ros2 run project_tasks dispatch_patrol -n 10 -p waypoint1
tinyRobot1_charger -F TinyRobot -R tinyRobot1 --use_sim_time
```

- This creates a patrol task that will wander 10 times between the points waypoint1 and tinyRobot1_charger and is assigned to the rinyRobot1 of type TinyRobot, remember to add `--use_sim_time` as this executes the task in simulation time, which is different from the computer time. This can be disabled in the launch files, if you do, remove the `--use_sim_time` option so the task is run in the proper time.

10. Add models to the environment

- You can also add models like furniture, decorations, object and more through the “add model” button or by pressing ‘O’. This will open a pop-up where you can search the item you want to place.

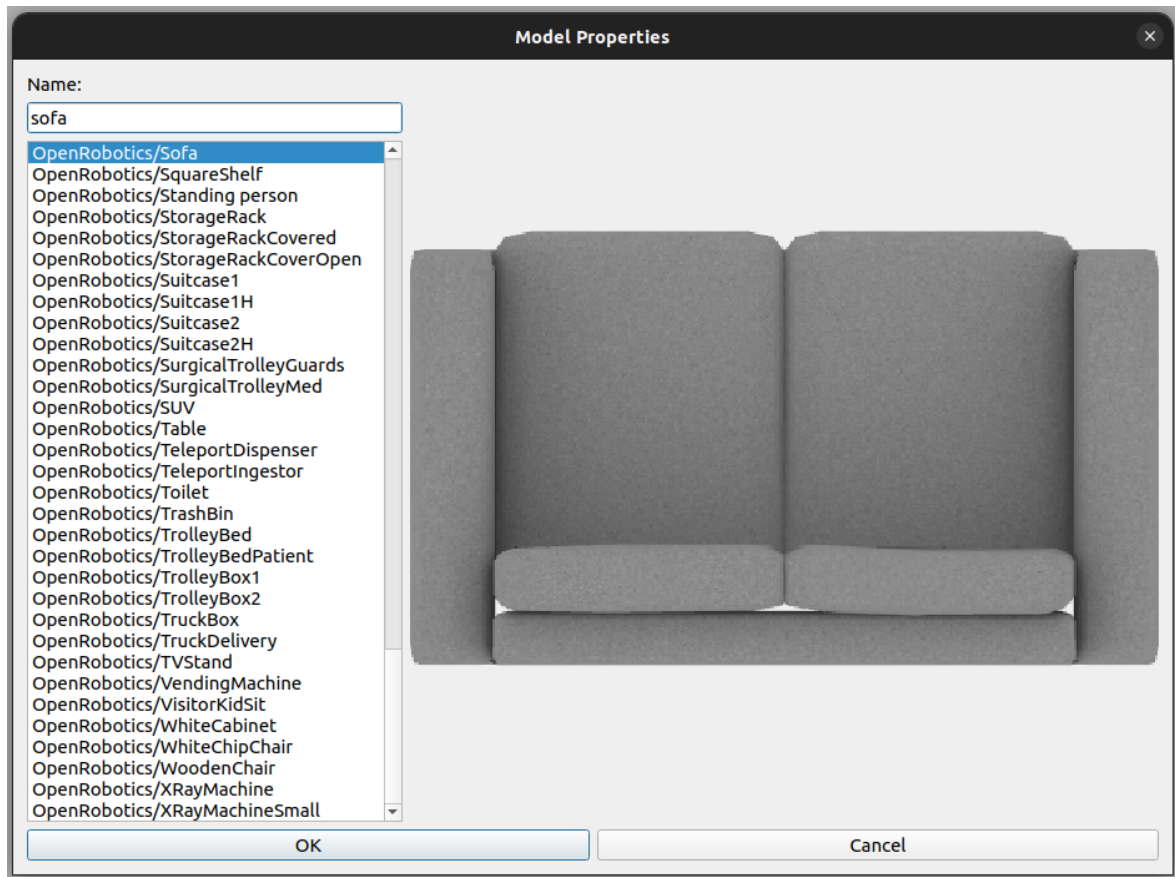


Figure 15: Select a model

- Now you can place it and use the rotate 'R' and move 'M' functionalities to adjust the orientation and position of the object you place. If you place multiple objects they can all be rotated going to 'Edit' -> 'Rotate all models'. rotation.

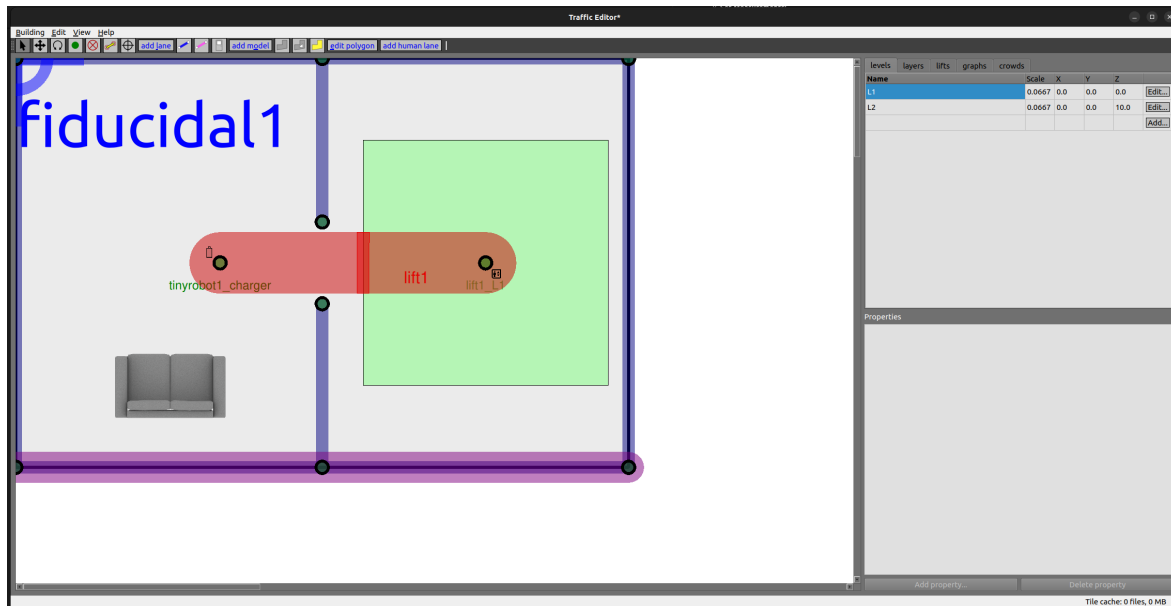


Figure 16: Place models

- Run the simulation and check the results, you should have something similar to Figure 17.

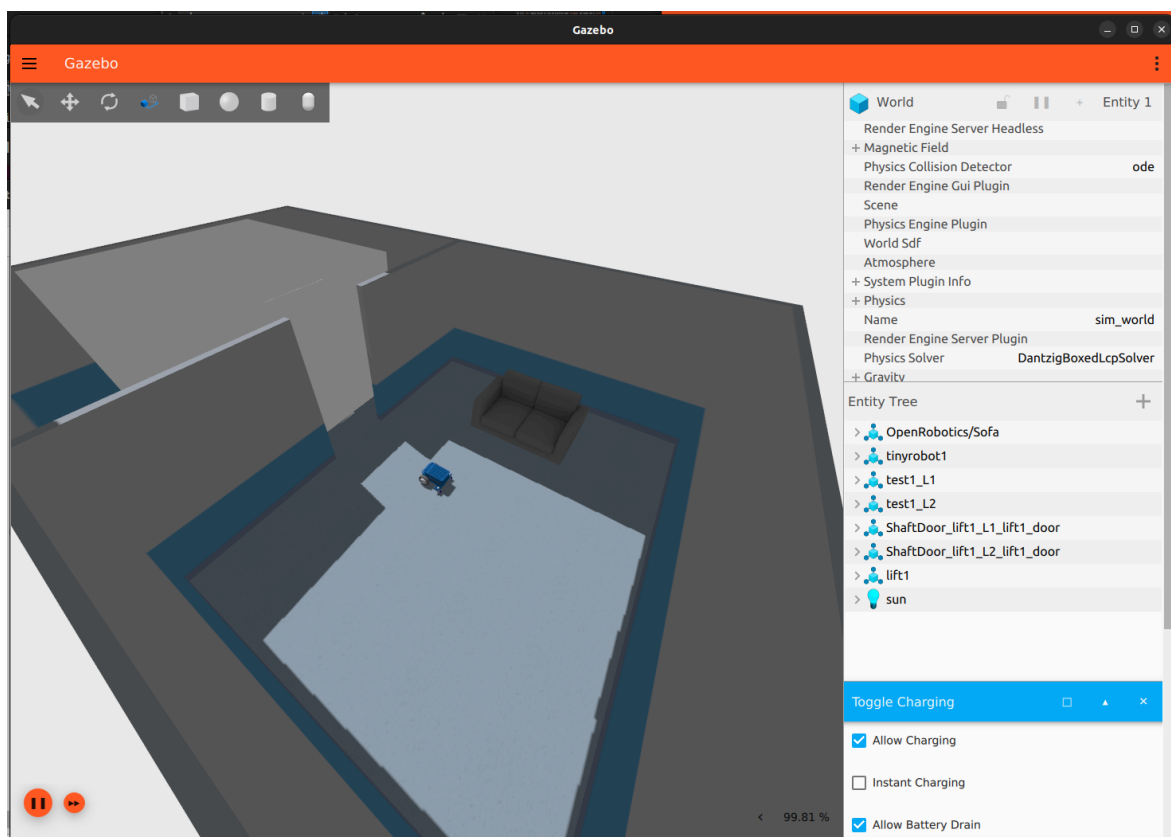


Figure 17: Simulation with models

11. Create Multiple Floors:

- To finish explaining all the tools, we will add a new floor and connect the floors using a lift. In this example, the doors have been removed(Figure 18).

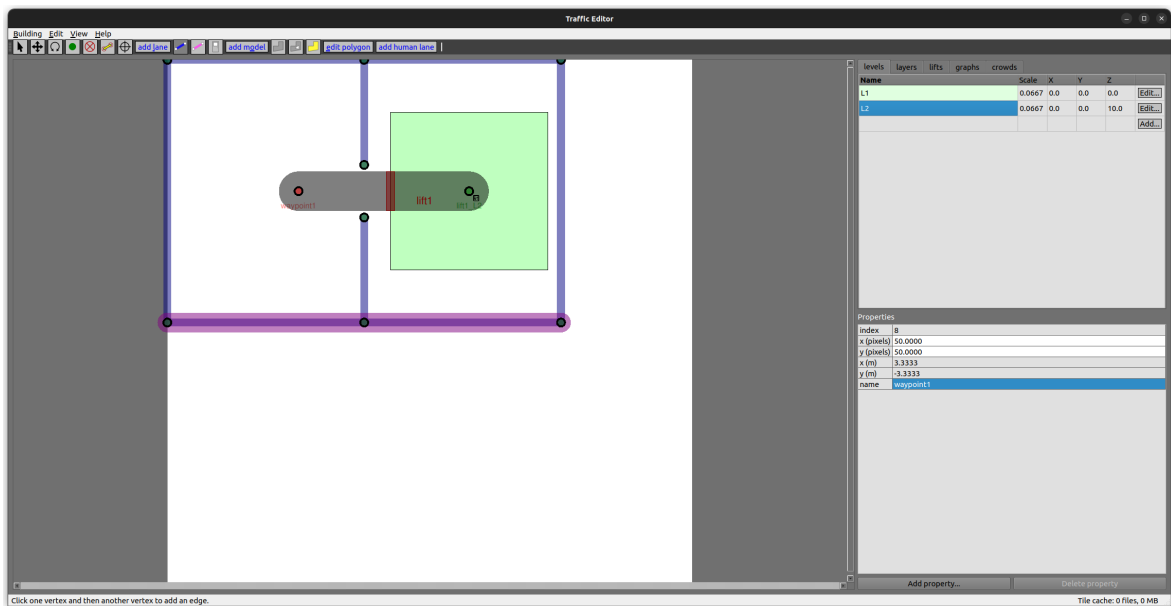


Figure 18: Second Floor

- To create a lift, go to the upper-right panel named lifts and add a new one. Fill in the information such as name, x and y positions, floors, and add one or multiple doors. This is a complex process as you need to align the door to the lift and the map and it may take some trial and error to get the result you want. Once you have the results you want, press “add lift waypoints” as shown in the example so that waypoints for each of the lifts floors are automatically created and then name them, check Figure 19 as an example.
- Try not to move this waypoints because the map generation depends on them, if you move them, click again on “add lift waypoints” to restore them.

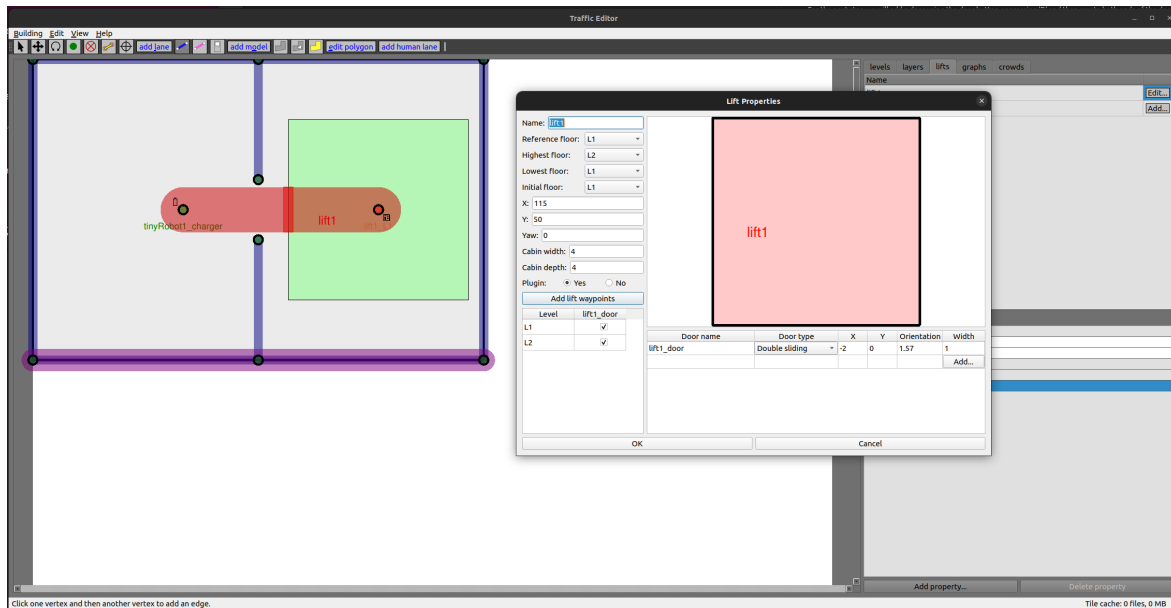


Figure 19: Creating a Lift

5.4. Execute tasks

In order to move the robots we need to execute different tasks, for example, we can order a robot unit to execute a patrol between different waypoints with the command:

```
ros2 run project_tasks dispatch_patrol -n 10 -p waypoint1 waypoint2
waypoint2 -F robot_fleet -R robot_name --use_sim_time
```

In this command we use the “-n” option to repeat the patrol 10 times, the “-p” option to indicate the different waypoints that compose the patrol’s path, the “-F” option to select a fleet of robots that should take this task, the “-R” to name the concrete robot that we want to execute the task with and “--use_sim_time” that, as explained before, launches the task using the simulation time and not the computer time.

There are multiple tasks in the “project_tasks” module like “goto” and “loop”, there is also a task to use teleop and directly control the robot and many others.

Some times it is hard to see what is wrong with the system and why something is not working, for example, when executing the task the request is not processed and we get the message “Did not get a response”. This may be caused by a faulty configuration, always check you are configuring the proper navigation graph, you must also check that you are not misspelling the robot name, the waypoint name, the fleet name or any other attribute of the task. To check if the robot is working you can use the “simple_api_server” that comes with OpenRMF demos, you can access localhost:8083/robot_list and this will display all the robots currently working in the system (Figure 20).

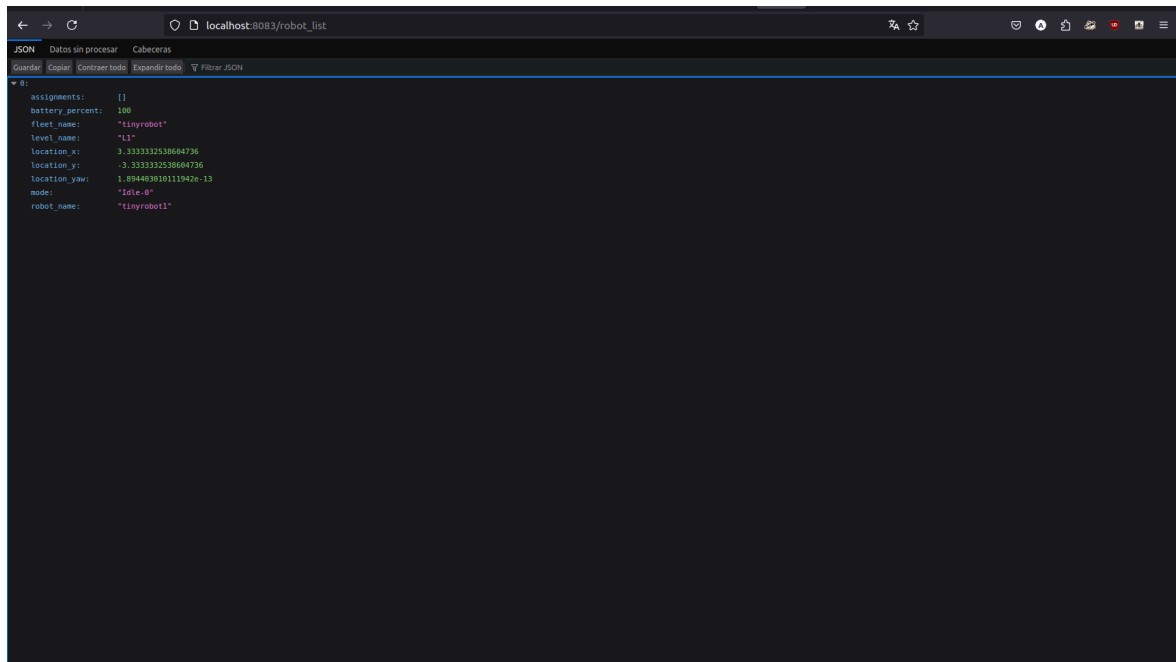


Figure 20: Simple API server displaying a list of robots

I encountered a problem, the task was not executing so I checked the “simple_api_server” and found out that the robot fleet was not able to launch properly since there was no robot listed there, after fixing the nav_graph to point to the proper .yaml file and restarting the simulation I was able to fix it. If you ever encounter this problem check the folder on “build/project_maps/maps/test1/nav_graphs”, the “number.yaml” that appears here should be the one in the “project_config/launch/test1.launch.xml”. If you find multiple number.yaml files it means you created multiple navigation graphs, if this is not intended try to fix it in the “traffic_editor” by deleting the lanes in a different color than the rest.

Now we execute the task again and the robot responds and starts moving as seen in Figure 21.

```

ros2 run project_tasks dispatch_patrol -n 10 -p lift1_L1
tinyrobot1_charger -F tinyrobot -R tinyrobot1 --use_sim_time

```

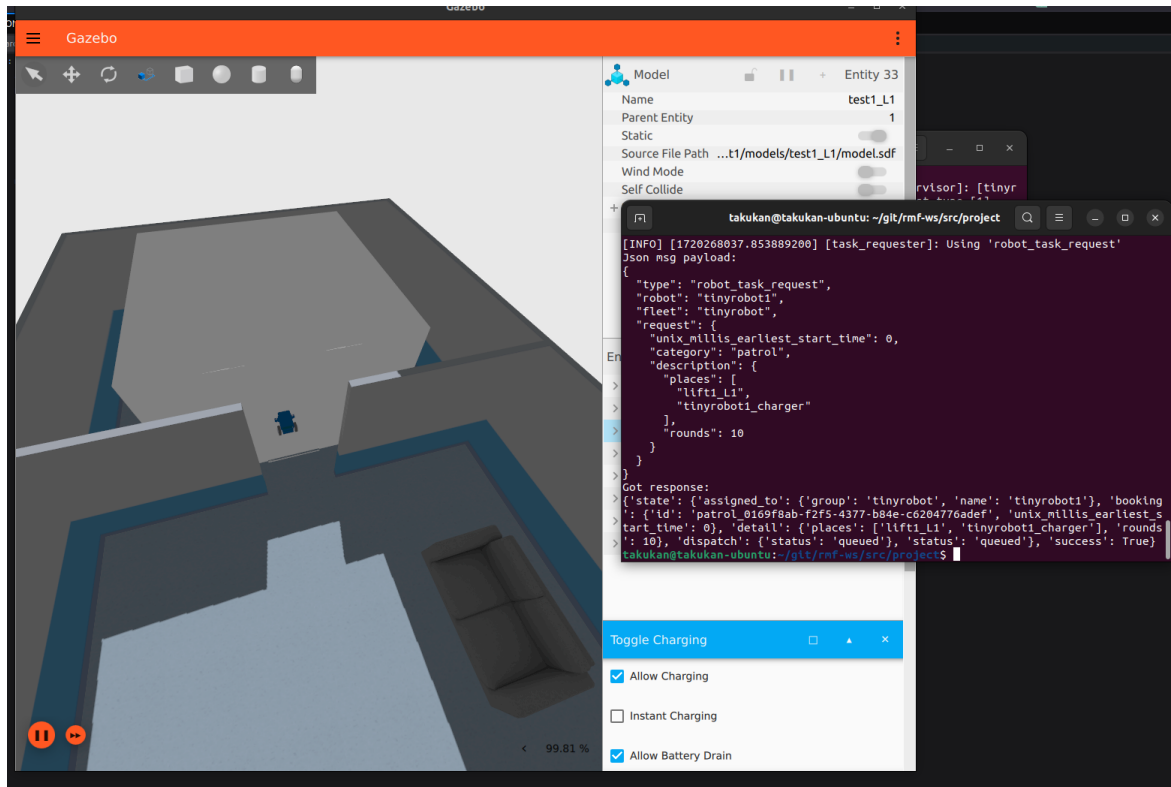



Figure 21: Robot moving after executing a task



Figure 23: EPS1 Building first floor



Figure 24: EPS1 Building second floor

- We created every floor with its own blueprint and placed the walls, doors and lift to simulate the eps1 scenario as in Figure 25

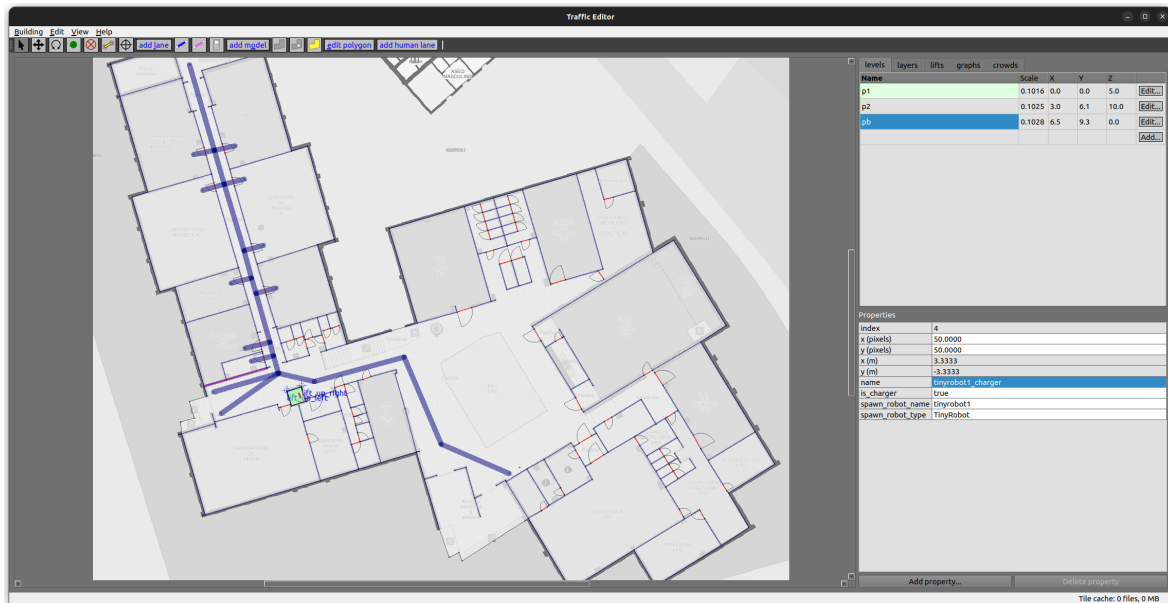


Figure 25: EPS1 Building

- Now we add some models to the map like tables and chairs but we find that rotating all the assets is not suitable for this task as different object need different rotations, in this cases you can edit manually the .yaml file of the building, if you go to the “models” section you will see you assets, there you can copy the “yaw” parameter that you already rotated the proper way and paste it in any other model that needs the same orientation. An example, we created a bunch of tables already properly rotated but now we need to add the chairs, we first place all the chairs that need to face one direction and then place the others(Figure 26).

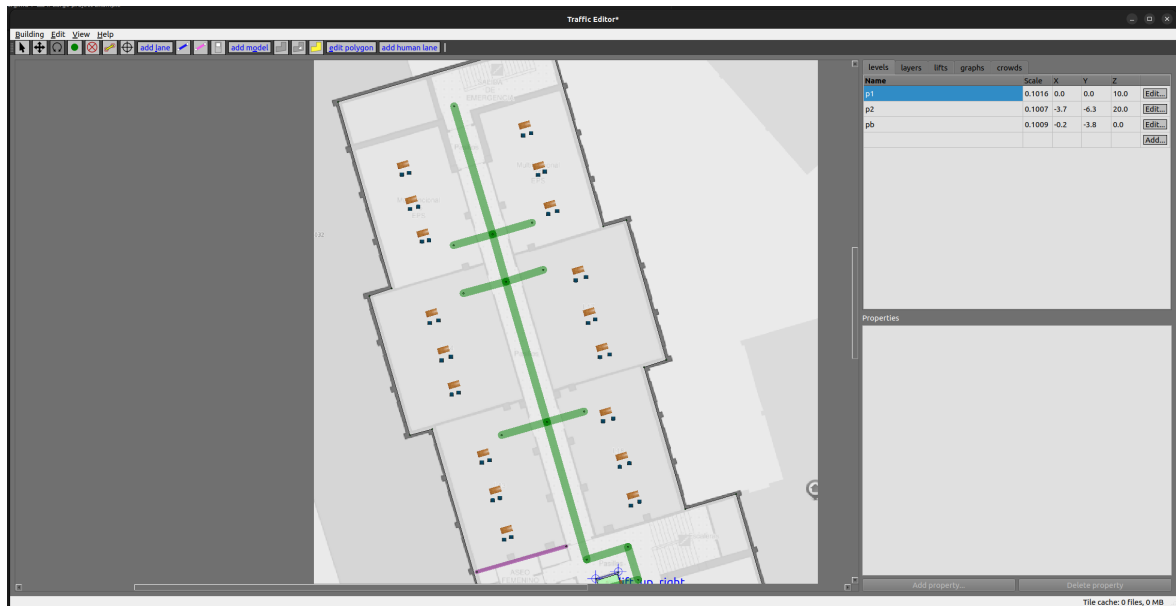


Figure 26: First row of chairs

- Then, we place the chairs that go the other way and we rotate only the first two that we placed, this way we make sure that in the yaml file the first chair of each set has the proper “yaw” parameter to copy it to the others, compare the Figures 26 and 27 to see the changes.

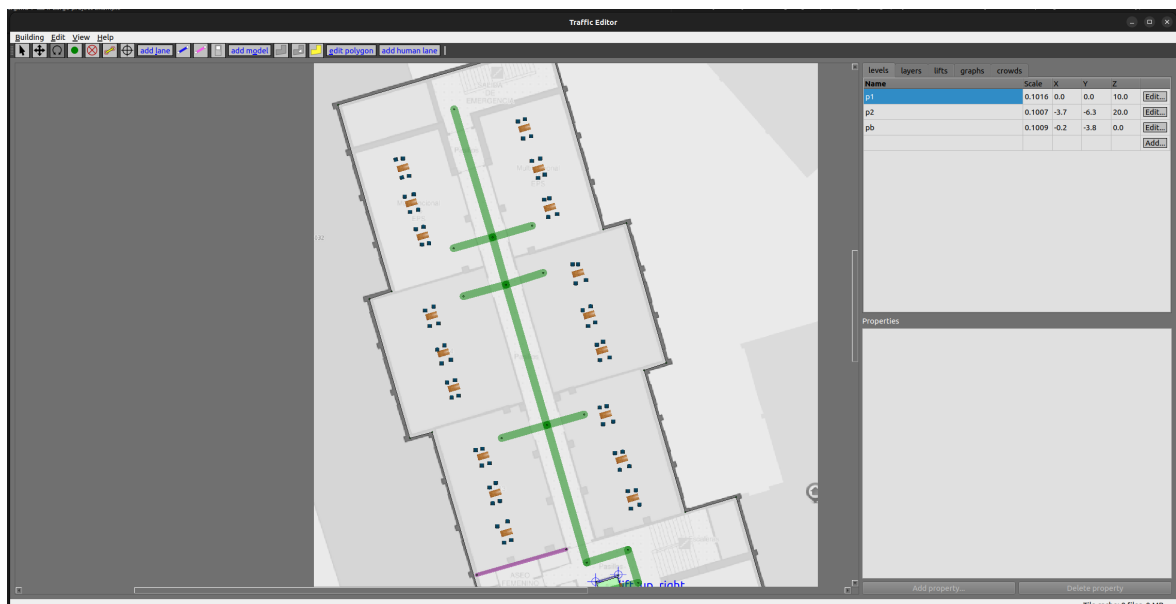


Figure 27: Contrary row of chairs

- See in the image below the first batch of chairs that we placed highlighted, we can notice this because it starts after the set of tables that we put before. The first chair has a different “yaw” than the others, so we can mass select them and change the value to the first one. We can do the same for the second batch of

chairs that starts with the first chair having a different “yaw” than the others as well.

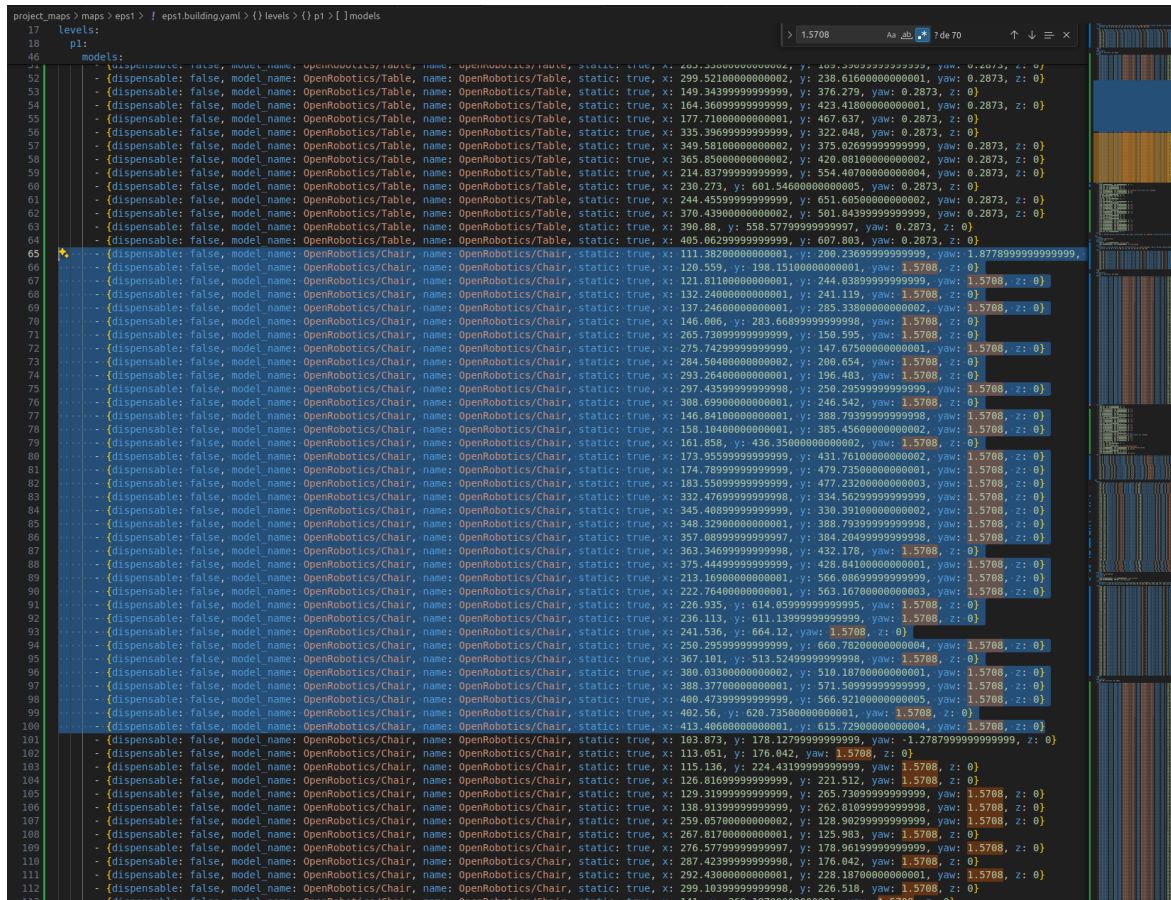


Figure 28: Yaml file with chairs highlighted

- After changing and saving we can reload the “traffic_editor” and, if done correctly, all the chairs will be facing the proper direction.



Figure 29: Chairs aligned

- Now we just need some final touches and run the simulation to check the result in Figure 30.

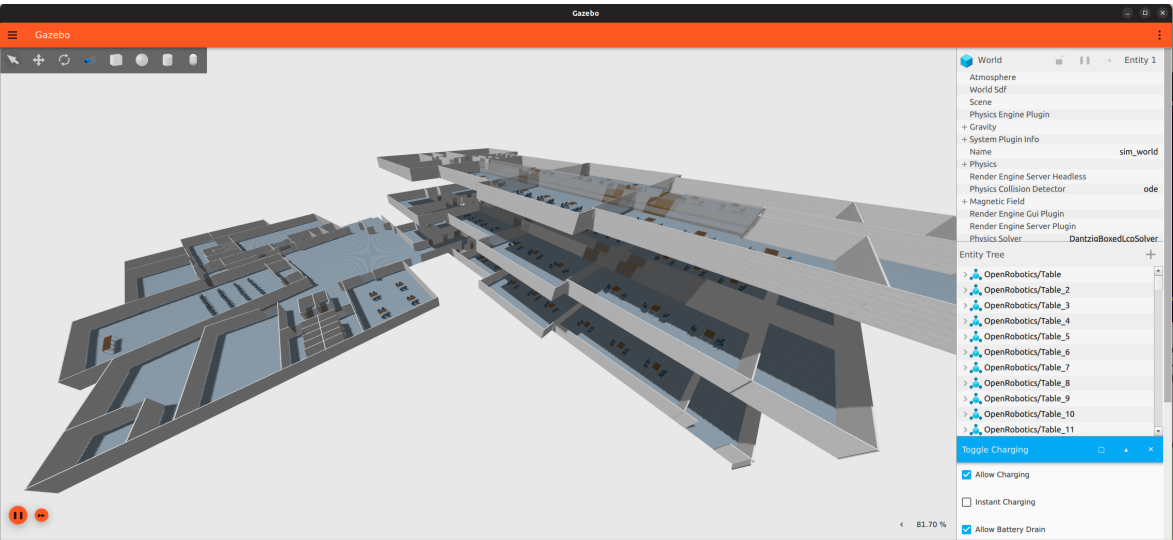


Figure 30: Final result

7. Conclusion

In this project, we detailed the complete implementation of the Robotics Middleware Framework (RMF) for multi-robot systems (MRS), showing important progress in the area of multi-robot coordination and management. Setting up RMF was both complex and demanding, requiring a lot of time and effort, especially for those new to the framework.

The system's complexity is significant. It includes many components and dependencies, each playing a crucial role in making sure the multi-robot fleets operate smoothly. The initial setup process, with its detailed installation instructions and scenario configurations, was vital to build a strong foundation for the system. This approach ensures that all parts are correctly set up and working as they should, thus reducing potential issues in future scenarios.

One of the main challenges was the coordination and communication among different types of robots. RMF's ability to bring together various robot models and simplify their operations within predefined environments was very helpful. The framework's support for different communication protocols and its strong task management features greatly improved the system's efficiency and reliability. However, it takes time to learn everything, making the learning process difficult and error debugging very tiresome.

The use of Python for scripting and automation within the ROS ecosystem further showed the flexibility and power of the language in developing complex algorithms and managing communications between different components, even though it may not perform as well as C++.

The successful implementation of RMF in this project has shown it to be a reliable and efficient middleware solution for MRS. It addresses current technological challenges and provides practical solutions that open the door for future experimentation, like creating fully automated environments from a simple building blueprint. The documentation and practical examples provided in this guide will help future users start a project in OpenRMF more easily.

In conclusion, while the initial setup of RMF requires a lot of time and effort, the benefits it brings to multi-robot coordination and management are significant. This project has laid a strong foundation for future developments, showing that with the right tools and methods, the complexities of multi-robot systems can be effectively managed. The advancements achieved help the ongoing growth of robotics, leading to greater efficiency, reliability, and innovation in the deployment of multi-robot systems.

8. Improvements

While this guide provides a comprehensive introduction to setting up and configuring RMF for a simple scenario, there are numerous advanced topics and enhancements that can be explored in future articles. These include creating custom robot models, adding existing robots to the system, and integrating robots with object-carrying capabilities. Here are some potential areas for future improvements and extensions:

8.1. In depth guide

This guide contains basic information about installation and set-up process to start up your project and ensure everything works fine, how to create your first map and start the simulation. There are other sections that could not be covered:

- **Creation of task to be performed by the robots**
 - Use existing python scripts or create new to execute tasks such as moving a robot, indicating a path to travel, looping, cleaning, etc...
 - Generate dashboard information to be used by rmf-web in order to automate or facilitate the execution of tasks
- **In depth guide**
 - Generating more concrete guides to showcase specific scenarios like multiple lifts, tasks priorities, blocked paths
 - Creating new modules for the system like grab and drop, service request and free roaming

8.2. Generating Your Own Robot Model and Specification

Creating a custom robot model allows for precise control over the robot's physical and functional characteristics. This involves defining the robot's geometry, sensors, actuators, and behaviors.

- **Defining Geometry and Physical Properties:**
 - Use URDF (Unified Robot Description Format) or SDF (Simulation Description Format) to define the robot's shape, size, and physical attributes.
 - Specify the robot's mass, inertia, and collision properties.
- **Integrating Sensors and Actuators:**
 - Add sensors such as cameras, LIDARs, and IMUs to the robot model.
 - Configure actuators for movement and interaction with the environment.
- **Customizing Robot Behavior:**
 - Define control algorithms and behaviors using ROS2 nodes.
 - Implement navigation, obstacle avoidance, and task-specific behaviors.

8.3. Adding Existing Robots

Integrating existing robots into the RMF ecosystem can enhance system capabilities and facilitate collaboration between different robot types. This could highlight MRS advantages by using more heterogeneous distributions

of robots.

- **ROS2 Integration:**
 - Utilize existing ROS2 packages and drivers to integrate popular robots such as TurtleBot, ClearPath robots, and drones.
 - Configure communication and control interfaces to ensure seamless interaction with RMF.
- **Fleet Management:**
 - Add multiple robot types to the RMF fleet management system.
 - Customize fleet adapters to handle different robot capabilities and task assignments.
- **Coordination and Cooperation:**
 - Implement coordination strategies to manage interactions between heterogeneous robots.
 - Utilize RMF's bidding and scheduling mechanisms to optimize task distribution.

8.4. Adding Robots with Object-Carrying Capabilities

Robots that can carry objects open up a wide range of applications, from logistics and delivery to warehouse automation and healthcare support. Although OpenRMF already contains an experimental mechanism to grab and drop objects it is, at the moment, very restricted and some improvements will become useful for future designs.

- **Defining Object-Carrying Robots:**
 - Design robots with grippers, trays, or other mechanisms for carrying objects.
 - Specify payload limits and handling characteristics in the robot model.
- **Task-Specific Behaviors:**
 - Implement behaviors for picking up, transporting, and delivering objects.
 - Use ROS2 action servers to manage object-handling tasks.
- **Integration with RMF Tasks:**
 - Configure RMF to assign delivery and transport tasks to object-carrying robots.
 - Utilize RMF's task dispatcher to manage object-handling workflows efficiently.

8.5. Exploring Advanced Scenarios

Although advanced simulation scenarios already exist within the RMF framework, further exploration and customization can provide deeper insights and more tailored solutions.

- **Multi-Floor and Multi-Building Scenarios:**
 - Create detailed maps with multiple floors and buildings.
 - Implement lifts and transitions between different levels.
- **Dynamic Environments:**
 - Simulate dynamic obstacles and changing environments.
 - Test robot adaptability and resilience to environmental changes.
- **Complex Task Coordination:**

- Implement and test complex task coordination strategies.
- Simulate real-world scenarios such as emergency response, search and rescue, and large-scale industrial automation.

9. References

- Ravankar Abhijeet et al. 2019. "Multi-Robot Task Allocation." MDPI.
<https://www.mdpi.com/2076-3417/9/13/2753>.
- Rongye Shi et al. 2019. "Path Planning for Multi-Robot Systems." MDPI.
<https://www.mdpi.com/2076-3417/9/19/4037>
- Dirk Thomas. n.d. "Quick Start Guide - Colcon Documentation."
<https://colcon.readthedocs.io/en/released/user/quick-start.html>
- FIPA. n.d. "Live Systems Resources - Fipa."
<http://www.fipa.org/resources/livesystems.html>
- IBM. n.d. "Common Object Request Broker Architecture (Corba)."
<https://www.ibm.com/docs/es/integration-bus/10.0?topic=corba-common-object-request-broker-architecture>
- Jesús Roldán-Gómez, Antonio, Juan \& Barrientos. 2021. "Special Issue on Multi-Robot Systems." <https://digital.csic.es/bitstream/10261/257093/1/Special%20issue%20on%20multi-robot%20systems.pdf>
- Juhásová Bohuslava, Martín Juhas \&. 2021. "Multi-Robot Systems for Industrial Applications." MDPI. <https://www.mdpi.com/2076-3417/10/15/5165>
- Kurt, Geihs. 2020. "Towards Intelligent Robotic Systems." MDPI.
<https://www.mdpi.com/2076-3417/10/4/1368>
- Meseguer Valenzuela, Andrés \& Blanes Noguera, Francisco. 2023. "State of the Art in Robot Fleet Management: From Task Allocation to Motion Planning."
https://ruc.udc.es/dspace/bitstream/handle/2183/33658/2023_Meseguer_Valenzuela_Andres_State_of_the_Art_in_Robot_Fleet_Management.pdf
- OpenRMF. n.d. "RMF Demos." https://github.com/open-rmf/rmf_demos
- OpenRMF. n.d. "RMF Simulation." https://github.com/open-rmf/rmf_simulation
- OpenRMF. n.d. "RMF Traffic Editor." https://github.com/open-rmf/rmf_traffic_editor
- OpenRMF. n.d. "RMF Visualization." https://github.com/open-rmf/rmf_visualization
- OpenRMF. n.d. "RMF Web Interface." <https://github.com/open-rmf/rmf-web>
- OpenRMF. n.d. "Robotics Middleware Framework (Rmf)."
<https://github.com/open-rmf/rmf>
- OpenRMF. n.d. https://github.com/open-rmf/rmf_demos/issues/177
- OpenRMF. n.d. https://github.com/open-rmf/rmf_demos/issues/169

- Open Robotics. n.d. "Beginner Client Libraries - Colcon Tutorial."*
<https://docs.ros.org/en/iron/Tutorials/Beginner-Client-Libraries/Colcon-Tutorial.html>
- Dirk Thomas. n.d. "Changes in Ros 2." <http://design.ros2.org/articles/changes.html>*
- Open Robotics. n.d. "Launch File Formats - Ros 2 Documentation."*
<https://docs.ros.org/en/iron/How-To-Guides/Launch-file-different-formats.html>
- Open Robotics. n.d. "ROS." <https://www.ros.org/>*
- Open Robotics. n.d. "ROS 2 Installation on Ubuntu."*
<https://docs.ros.org/en/iron/Installation/Ubuntu-Install-Debians.html>
- Open Robotics. n.d. "ROS 2 Installation Troubleshooting."*
<https://docs.ros.org/en/iron/How-To-Guides/Installation-Troubleshooting.html>
- Open Source Robotics Foundation. n.d. "Traffic Editor - Programming Multiple Robots with Ros 2." <https://osrf.github.io/ros2multirobotbook/traffic-editor.html>*
- Palafox, Pablo et al. 2019. "Coordination in Multi-Robot Systems." MDPI.*
<https://www.mdpi.com/2076-3417/9/13/2661>
- Panda3D Community. n.d. "Setting up Powerful Debugging with Ipdb - Panda3d Discourse." <https://discourse.panda3d.org/t/tut-setting-up-powerful-debugging-with-ipdb/12616>*
- Python Software Foundation. n.d. "The Python Debugger (Pdb)."*
<https://docs.python.org/3/library/pdb.html>
- Roderich Groß, Edward Bray \&. 2023. "Advanced Robotics for Smart Manufacturing." SpringerLink. <https://link.springer.com/article/10.1007/s43154-023-00106-y>*
- Xavier Défago, Keisuke Okumura \&. 2023. "Advances in Artificial Intelligence for Robotics." ScienceDirect.*
<https://www.sciencedirect.com/science/article/abs/pii/S0004370223000929>.